



Stress Testing Control Loops in Cyber-physical Systems

CLAUDIO MANDRIOLI and SEUNG YEOP SHIN, University of Luxembourg, Luxembourg

MARTINA MAGGIO, Saarland University, Germany and Lund University, Sweden

DOMENICO BIANCULLI, University of Luxembourg, Luxembourg

LIONEL BRIAND, University of Luxembourg, Luxembourg and University of Ottawa, Canada

Cyber-physical Systems (CPSs) are often safety-critical and deployed in uncertain environments. Identifying scenarios where CPSs do not comply with requirements is fundamental but difficult due to the multi-disciplinary nature of CPSs. We investigate the testing of control-based CPSs, where control and software engineers develop the software collaboratively. Control engineers make design assumptions during system development to leverage control theory and obtain guarantees on CPS behaviour. In the implemented system, however, such assumptions are not always satisfied, and their falsification can lead the loss of guarantees. We define stress testing of control-based CPSs as generating tests to falsify such design assumptions. We highlight different types of assumptions, focusing on the use of linearised physics models. To generate stress tests falsifying such assumptions, we leverage control theory to qualitatively characterise the input space of a control-based CPS. We propose a novel test parametrisation for control-based CPSs and use it with the input space characterisation to develop a stress testing approach. We evaluate our approach on three case study systems, including a drone, a continuous-current motor (in five configurations), and an aircraft. Our results show the effectiveness of the proposed testing approach in falsifying the design assumptions and highlighting the causes of assumption violations.

CCS Concepts: • **Software and its engineering** → **Software verification and validation**; • **Computer systems organization** → **Embedded and cyber-physical systems**;

Additional Key Words and Phrases: Cyber-physical systems, software testing, control theory

ACM Reference format:

Claudio Mandrioli, Seung Yeop Shin, Martina Maggio, Domenico Bianculli, and Lionel Briand. 2023. Stress Testing Control Loops in Cyber-physical Systems. *ACM Trans. Softw. Eng. Methodol.* 33, 2, Article 35 (December 2023), 58 pages.

<https://doi.org/10.1145/3624742>

C. Mandrioli part of this work was done while the author was a PhD student at Lund University, Sweden.

This project has received funding from the European Union's Horizon 2020 research and innovation programme under Grants No. 871259 (ADMORPH project) and No. 957254 (COSMOS); from NSERC of Canada under the Discovery and CRC programs; from the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation. This publication reflects only the authors' view and the European Commission is not responsible for any use that may be made of the information it contains.

Authors' addresses: C. Mandrioli (Corresponding author), S. Y. Shin, and D. Bianculli, University of Luxembourg, Avenue J.F Kennedy 29, L-1855 Luxembourg, Luxembourg; e-mails: {claudio.mandrioli, seungyeob.shin, domenico.bianculli}@uni.lu; M. Maggio, Saarland University, Saarbrücken Campus, 66123 Saarbrücken; e-mail: maggio@cs.uni-saarland.de; L. Briand, University of Luxembourg, Avenue J.F Kennedy 29, L-1855 Luxembourg, Luxembourg; University of Ottawa, 800 King Edward Avenue, Ottawa, ON K1N 6N5, Canada; e-mail: lionel.briand@uni.lu.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivs International 4.0 License.

© 2023 Copyright held by the owner/author(s).

1049-331X/2023/12-ART35

<https://doi.org/10.1145/3624742>

ACM Transactions on Software Engineering and Methodology, Vol. 33, No. 2, Article 35. Publication date: December 2023.

1 INTRODUCTION

Cyber-physical Systems (CPSs) are engineering artefacts characterised by the tight coupling of physical and software components [31]. This tight coupling is created by sensors and actuators that allow the software component to measure and affect the physical part of the system. For example, a drone uses sensors (such as accelerometer and camera) to estimate its position and actuators (such as propellers) to move. The desired behaviour is usually that the drone performs stable flight and reaches a desired position.

CPSs are often by nature safety-critical, and they are expected to operate in uncertain environments [54]. For example, drones and cars operate in environments where people are present and the external conditions are never fully known (e.g., the presence of wind and obstacles for drones, and other vehicles and pedestrians on the road for cars). In such circumstances, it is of primary importance to identify the scenarios in which the CPS is no longer able to fulfil its requirements. To identify such scenarios, *stress testing* aims to execute a **system under test (SUT)** in conditions that are different from the ones expected during the system design [46]. However, CPS development is known to be multidisciplinary [31], and the definition of the operating conditions expected during the CPS design depends on the combination of design choices made by different types of engineers (in our case, software engineers and control engineers). For example, for a drone, we want to identify what can limit its capability to avoid a moving obstacle. The limiting factors can be the hardware design (e.g., the size of the propellers), different software components (e.g., the path planning), or their interaction.

Intuitively, being composed of a physical and software part, CPS development involves both software engineers and engineers with specific knowledge of the physical part of the system (e.g., aerospace engineers for a drone or mechanical engineers for a car). However, besides these categories of engineers, many CPS applications involve also control engineers [26]. We call CPSs that involve control engineers *control-based CPSs*.¹ In such applications, there are multiple layers of decision making, depending on the decision's level of abstraction and time scale. Among the different layers, we highlight in Figure 1 the role of the *control layer* in the interaction between the cyber component (green dashed box) and the physical component (purple dashed box). The control layer receives desired values (the *references*) for physical quantities of interest and uses sensors and actuators in real-time to enforce these values in the physical component. Accordingly, CPS requirements are generally defined over quantities that live in the physical part of the system, captured by the *CPS outputs* in the figure. For example, for a delivery drone, high-level decision making concerns the definition of the sequences of picking up the object to be delivered, while the control layer receives the desired position, reads the sensors values, and adjusts in real time the propellers commands to reach and maintain the desired position.

In control-based CPSs, the role of control engineers is to design *control algorithms* that are implemented in software as part of the control layer. When designing control algorithms, control engineers leverage control theory. To apply such theory, they make *design assumptions* and abstract certain aspects of the design problem. Such abstractions concern both the physical component as well as the software implementation of the control layer. For example, for a drone, they simplify how the drone moves in space, they neglect the finite capacity of the motors to generate thrust (also known as actuator saturation) and dismiss other functionalities of the software like flight mode changes.

¹CPSs developed with control theory have also been called “control-CPSs” in the literature [26]. We believe “control-based CPSs” is a more precise characterisation, since the word “control” can have different meanings depending on the context (e.g., control in the sense of supervision).

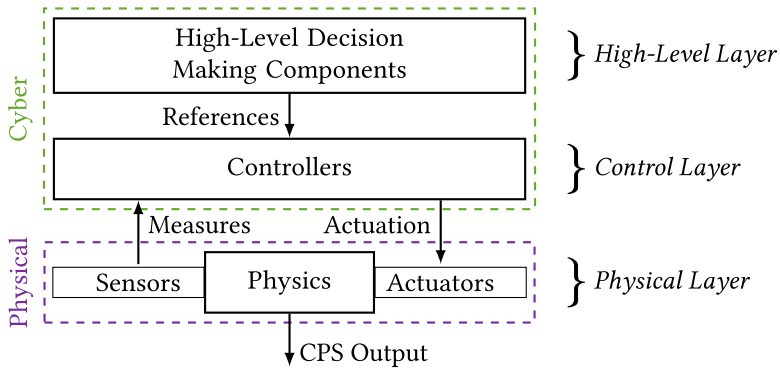


Fig. 1. Structure of a CPS, where a cyber component (green dashed box) interacts with a physical component (purple dashed box). In the cyber component, we highlight the controllers that implement the control layer and handle the low-level interaction with the physics. The control layer receives desired values for given physical quantities (the *references*) and uses sensors and actuators to enforce these values in the physical component. The measures and actuation signals create a closed-loop interaction between the control layer and the physical layer. In fact, CPS requirements are generally defined over quantities that live in the physical part of the system, captured by the *CPS outputs*.

From a theoretical point of view, control theory provides formal guarantees, under its design assumptions, on the CPS performance (e.g., drone flight speed along a certain trajectory). In practice, the design assumptions do not necessarily hold for the implemented CPS, and the guarantees might be lost. We can then distinguish scenarios (e.g., values of the references, environment conditions, system state) in which the design assumptions hold (and therefore also the formal guarantees are valid) from scenarios in which they are *falsified* (and the behaviour of the system is unpredictable). In general, a CPS is expected to be robust to some level of falsification of a design assumption (e.g., if an assumption is falsified for a limited amount of time) and be still able to provide the guarantees. Following this observation, we define the *design scope* of a CPS as the set of scenarios in which the design assumptions either hold for the CPS implementation or their falsification does not impact the CPS behaviour. Conversely, we consider scenarios where the software implementation of the control layer is affected by the falsification of the design assumptions to be outside of the design scope of the CPS. In other words, the falsification of design assumptions limits the CPS design scope, i.e., it reduces the set of scenarios in which the designed algorithm can provide *a priori* guarantees. For example, in the drone, scenarios in which the actuators do not saturate (i.e., they do not reach their maximum or minimum power), or saturate without affecting the flight, are within the design scope. Conversely, scenarios in which the limited capacity of the motors limits the drone flight capabilities are outside of the design scope.

We argue that, when performing the verification and validation of a control-based CPS, software engineers have to take into account its design scope, which is limited by the falsification of the design assumptions in practice. In fact, when the CPS is executed within the design scope, the control software can be expected to have predictable behaviour thanks to the *a priori* guarantees (although other types of faults can of course still appear). In contrast, when the CPS is outside of the design scope, the control layer can expose unpredictable behaviour. To give a practical example, a delivery drone can be led out of its design scope by a specific sequence of reference values fed to the control layer, such as those created by unexpected obstacles encountered during its mission. When this happens, the ability of the drone to perform stable flight is impaired and the delivery is likely compromised. By identifying the design scope of the CPS, software engineers can decide, for

example, if the CPS needs changes in the design, runtime checks, or fall-back solutions to ensure the CPS's stable and reliable operation.

Previous literature on control-based CPS testing focuses the testing process on the identification of test cases that lead the SUT to violate its requirements [40]. Accordingly, several prior works propose various approaches to CPS test case generation based on metrics that quantify the fulfilment of the requirements (e.g., approaches based on search algorithms [37], or classification trees [30]). In this article, we propose a complementary perspective to the test case generation problem. Instead of focusing on testing the requirements, we focus on testing to falsify the control design assumptions. The intuition is that, in the scenarios where such assumptions are fulfilled, we can rely on the *a priori* guarantees of control theory. Conversely, in the scenarios where they are not fulfilled, the behaviour of the CPS is unpredictable and empirical evaluation through testing is needed.

We use the falsification of control design assumptions to define the problem of stress testing control-based CPS software. In this testing problem, the objective is to generate test cases that falsify, to different degrees, the design assumptions (i.e., stress test cases) and identify when such falsification prevents the control algorithm from providing guarantees. We use knowledge from the control engineering domain to identify the different types of design assumptions that engineers make during the control design process. By excluding the assumptions that can be addressed with existing testing approaches, we focus this work on the assumptions related to the use of *linearised physics models* [3]. For generating test cases that falsify the linearised models, we use frequency-based control-theoretical models to qualitatively describe the input space of the control layer. To leverage this qualitative input space characterisation, we propose a novel approach for the definition of test cases (test case parametrisation) and a novel metric for identifying the stress test cases for control-based CPS. More specifically, we propose to define test cases according to (i) a shape function, (ii) an amplitude scaling coefficient, and (iii) a time-scaling coefficient. Using such parametrisation and metric, we develop a complete stress testing approach for the linearised model assumption.

We assessed our testing methodology by applying it to three different case studies: the altitude control of a drone, the position control of a DC servo (a continuous current motor), and aircraft altitude control. Our results suggest that our approach and metrics are capable of generating and identifying test cases that falsify to different degrees the control design assumptions. This in turn allows engineers to observe the behaviour of the SUT at the bounds of its design scope where *a priori* control-theoretical guarantees become uncertain. Most notably, we generate tests that expose, across our case studies, actuator saturation time ranging from 0% to 95%, thus allowing the evaluation of the CPS under a large variety of saturation conditions. Actuator saturation is arguably the most common source of non-linear behaviour that can falsify the linear models design assumption in CPSs [27]. This is due to actuators being one of the most expensive components in CPS and therefore being sized to the minimum to reduce costs. For this reason, they often saturate during CPS operation. Hence, assessing the ability of the CPS to operate in the presence of actuator saturation is of prime importance. In this article, based on case studies, we showcase how our tests can be used to gain insights into design choices that limit the ability of the SUTs to perform safely in different scenarios. For example, how actuator saturation can limit the trajectories that a drone can follow during flight.

To summarise, this article makes the following contributions:

- (i) We use control-theory domain knowledge to develop an input space qualitative characterisation (i.e., qualitatively describe the expected behaviour of the SUT according to different input features) for individual reference values, i.e., the real-valued inputs of the control layer (Section 4.1).

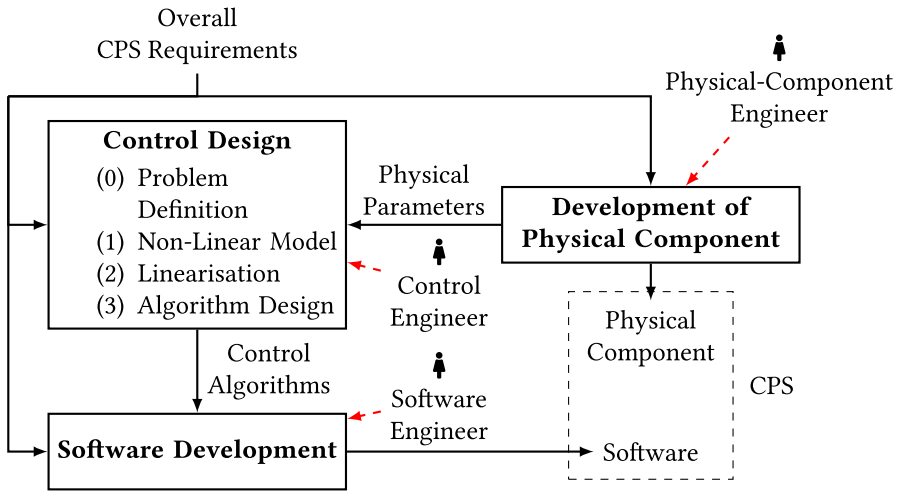


Fig. 2. Simplified high-level description of the control system development; the red dashed arrows represent development steps, while the black solid arrows represent data flow. The development flow is simplified (e.g., neglecting iterations) to focus on the role of control engineering.

- (ii) We define different metrics to quantify the relevant aspects of the proposed input space characterisation (Section 4.2).
- (iii) We propose a novel test case parametrisation for CPS control layers that enables the use of the characterisation (Section 5.1). We use such parametrisation to develop metamorphic relations for control-based CPSs and a control-based CPS stress testing approach (Section 5.3).

The rest of the article is structured as follows. First, given the multidisciplinary nature of this problem, we present in Section 2 the relevant background on the control-based CPS development and the control algorithms design process. Section 3 provides a control engineering perspective of CPS stress testing. In Section 4, we describe our characterisation of the reference values input space. We illustrate our testing approach in Section 5. Section 6 reports on our empirical evaluation. Section 7 discusses the applicability and limitations of the approach. Section 8 presents the related work on the testing process for cyber-physical systems and control systems in particular. Section 9 concludes the article and outlines directions for future work.

2 CONTEXT AND BACKGROUND

This section provides the relevant background on the development of control-based CPSs; we use the position control of a drone as running example to exemplify the different concepts. In such CPSs, the control layer receives desired values for the position of the drone, for example, the desired x , y , and z coordinates. It uses sensors to estimate the drone's current position and actuators to bring the drone to the desired one. The section is divided in two parts. First, in Section 2.1, we present the engineering process for developing a control-based CPS, and we define the key roles involved in this process. Then, in Section 2.2, we provide the relevant control engineering background needed to understand the remainder of this article.

2.1 Development of Control-based CPS

Figure 2 provides a graphical overview of the typical development workflow of control-based CPSs. This overview is simplified and focuses on the role of control engineers in the development of the software that handles the interaction between the physical and the cyber components.

The development of any engineering system starts with the definition of the system's requirements, denoted by *Overall CPS Requirements* in the figure. In the case of control-based CPSs, the requirements usually describe the desired behaviour of the *physical* part of the system. In our running example, the overall CPS requirements describe, at a high level, how the drone is supposed to move in space. A concrete example of such a requirement could be that the drone must plan and follow a prescribed trajectory (at every point in time) with no discrepancy higher than a given threshold (e.g., 1 m), and reach the desired position within a given time duration (e.g., 3 min).

These requirements are then made available to the engineers involved in the development of the CPS. We identify three types of engineers involved, each having different duties. In Figure 2, we use red arrows to highlight their role in the development.

- (i) *Physical-component Engineers* design the *physical part* of the system. These could be for example mechanical or aerospace engineers depending on whether the considered CPS is a car or a drone. In our running example, an aerospace engineer may size up the propellers and draw the mechanical structure of the drone's body.
- (ii) *Control Engineers* select and design the control algorithms, managing interactions between the physical and cyber parts. In the case of our running example, this means designing the algorithms in charge of estimating the drone's current position based on the sensors' readings, and computing the commands to be sent to the drone motors so that it can fly.
- (iii) *Software Engineers* are in charge of the cyber part of the CPS. They implement the control algorithms, and design the functions that are needed to integrate the different software components (e.g., the high-level decision making in Figure 1). In our running example, there might be an aggressive and faster control algorithm when the drone is flying outdoors, compared to a safer one when the drone is flying indoors in a constrained space.

In Figure 2, we use black arrows to highlight the data and information flow during the development workflow. For instance, in our running example, the physical-component engineer communicates the data related to the propeller thrust to the control engineer, who then uses it in the *control design step*, further described in Section 2.2. The result of the control design process is a set of algorithms that are provided to the software engineers, so that they can implement them and integrate them with the rest of the software necessary to fly the drone (e.g., the functions that perform initial checks, sensor data acquisition, communication with the motors). When software engineers implement and integrate control algorithms, they make different choices that can alter the behaviour of control algorithms. For example, a control algorithm can behave differently depending on how frequently and how much its inputs values (the references) are changed. Vice versa, the design of the control algorithm (e.g., how it reacts to reference changes) will affect which implementations of the software fulfill the requirements and which do not.

Consequences of Multidisciplinarity on the Testing Process: The development workflow highlights the multidisciplinary nature of control-based CPSs. More specifically, it highlights that control engineers play an essential role in developing control-based CPSs to bridge software implementation (i.e., software engineers' concern) and physical components development (i.e., physical-component engineers' concern). This multidisciplinarity impacts the testing of such systems [34]. In particular, when testing the CPS control layer, we must account for the control algorithms implemented in the control layer. Indeed, the control layer includes control algorithms designed by the control engineers which are however implemented and integrated with other functionalities (e.g., mode switches) by the software engineers. Furthermore, since requirements of CPS usually concern the behaviour of the physical component, testing must necessarily include the latter in the setup. In other words, testing the software in isolation has limitations, since it

does not account for the interactions between the software and physical components, which limit the design scope. Therefore, engineers must account not only for the software implementation but also such interactions, which requires multidisciplinary considerations, including both control and software engineering.

2.2 Control Engineering Primer

In this section, we introduce the definition of a control design problem and the control design process.² We illustrate the frequency-domain description of signals and systems (alternative to the time-domain description), and introduce the basic concepts used in the remainder of this article.

Definition of Control Design Problems: As mentioned in Section 1, the objective of the control layer in a CPS is to steer physical quantities to track desired values. More rigorously, the input of the control layer is the vector r of desired values (also called *references*). The control objective is to ensure that the actual values in the physical system are as close as possible to the corresponding reference values. Using the control terminology, we say these physical quantities constitute an *output* vector y , and the control objective is $y \approx r$ (i.e., y tracks the reference values in r).

To achieve its objective, the control layer uses sensors to iteratively measure signals from the physical part of the system, and actuators to steer it. Based on measurements and reference values, the control algorithm computes the commands to be sent to the actuators. The control algorithm is executed repeatedly, at constant time intervals, resulting in a continuous interaction between cyber and physical components. This interaction is called *control loop*. The control loop creates a cause and effect cycle for which the output of the control algorithm (the actuation) affects the physical components and therefore its own input (the sensor readings). Because of this cause and effect cycle, the cyber and physical components are said to be in *closed loop*.

In the drone example, the control objective is to use the propellers to move the drone following a reference trajectory. The software iteratively (i) uses sensors to measure quantities like its own acceleration every millisecond, (ii) executes the control algorithm, and (iii) actuates the motors spinning the propellers by sending voltage commands. On the physical side, the propellers generate forces that cause the drone movement, and in turn affect the future acceleration readings, leading to the closed-loop interaction.

As r and y are vectors, the engineers usually define multiple control loops, often one for each element of the vectors. In a drone, we can expect to find one loop for each of the three dimensions in the space that the drone can move in: forward or backward, left or right, and up or down. Furthermore, the CPS requirements might call for different control modes, such as fast (but risky) flight mode, and a safe (but slower) one. In the most common control design approaches (like proportional-integral-derivative control, also known as PID control), the design of the different control loops and control modes is addressed separately: engineers develop a dedicated control algorithm for each mode. The implementation can switch between the algorithms of the different modes during execution.

The identification of the control loops and modes is the *control problem definition*, and constitutes a preliminary step in the control design process. For each of the identified loops and modes, the control design problem is reduced to the design of the control algorithms needed to control the physics. Each control algorithm, when executed in closed loop with the physical component, is expected to guarantee (to the best degree possible) that the output y tracks the reference r when operating in the mode it is designed for. We now discuss the control design process for one individual loop and mode.

²The content of this section is mostly based on the book “Feedback Systems: an Introduction for Scientists and Engineers” [3].

Control Design Process: As highlighted inside the control design block in Figure 1, the design of a control algorithm comprises of three main steps. The first step is to define an equation-based model of the physical component. The role of this model is to provide a representation of how the actuators affect the measurements and output. For example, in the drone, this is a model that represents how the thrust generated by the propellers affects the position and orientation (called attitude) of the drone itself. These models can be obtained using either first-principle approaches, through the laws of physics, or with data-driven approaches through system identification [6]. Either way, the models are typically in the form of non-linear differential equations, i.e., non-linear equations that contain both signals and their derivatives (rate of change). Non-linear models are difficult to analyse, as small changes in the input can cause significantly different behaviours and hence they do not allow for approaches that apply with generality to a variety of systems [29].

To overcome the complexity of non-linear models, the second step of the control design is to approximate them using linearised models. Retrieving such approximated models is called *linearisation*, and restricts the model scope to the surroundings of an expected operating point.³ Accordingly, the operating point is chosen as the physical state around which we expect the system to operate most of the time. In the drone, the operating point would be the horizontal state in which the drone is parallel to the ground and not tilted in any direction. Through linearisation, we then obtain a model that is still valid for small variations in the attitude angles around the operating point. The model is now a set of ordinary linear differential equations and therefore it can be handled in a simpler way thanks to a large variety of analytical tools [3].

The third step is finally the control algorithm design. To design such algorithm, control engineers use control theory. To this end, they make different assumptions about the system (e.g., that the physical component is sufficiently close to the assumed operating point). For models based on linear differential equations, control theory provides numerous tools to perform exact analyses and design control algorithms with formal performance guarantees. Such tools are based on a frequency-domain description of the physical system. Frequency-domain descriptions are well-suited for treating ordinary differential equations, because they provide a compact description for the derivative of a signal with respect to the signal itself. This makes it easier to analyse the physics and draw conclusions on the system's properties. The control algorithms obtained using control theory are also in the form of linear differential equations.

Frequency-domain Descriptions: We now provide a high-level description of the frequency-domain for both signals and systems, together with the intuition of why the frequency-domain is well suited for analysing and manipulating differential equations.

The frequency-domain description of signals is based on the fact that signals can be decomposed and treated as the sum of sinusoidal functions with different frequencies. The description in the frequency-domain specifies which sinusoidal components are present in the signal and what their amplitude is. This is in contrast to the time-domain, where signals are represented as a sequence of values over time. The frequency-domain sinusoidal components are commonly called *frequency components*: for example, a fast-changing signal is mostly composed of fast (i.e., high-frequency) sinusoids. On the contrary, a signal that does not change much is mostly composed of slow (i.e., low-frequency) sinusoids.

The translation of a signal from the time-domain representation to its frequency-domain one uses the *Fourier Transform*—or its time-sampled equivalent **Discrete Fourier Transform (DFT)** [16], which we will use in the remainder of this article. Figure 3 shows three examples

³Linearising a non-linear equation means approximating its non-linear relations (e.g., if a variable is squared, like in the case of aerodynamic drag) with a linear function. The linear function is based on the first derivative of the non-linear relation, and more specifically, on the value of the first derivative in the chosen operating point.

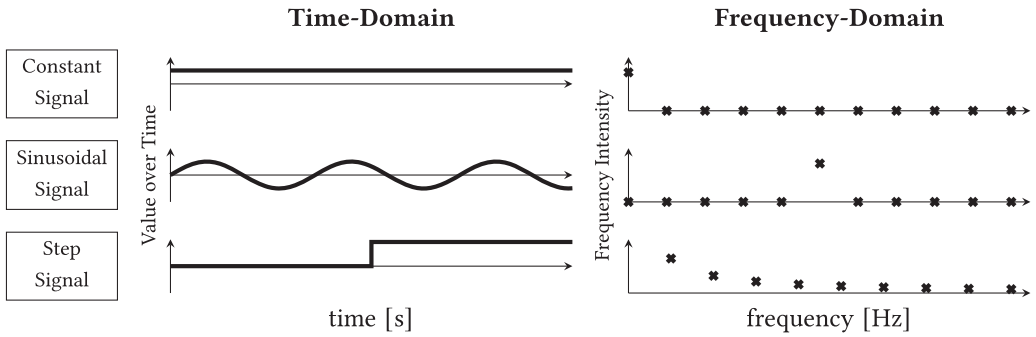


Fig. 3. Examples of DFT spectra (on the right) of different signals defined in the time-domain (on the left). A constant signal is described by only a zero frequency component, a pure sinusoidal maps to one single frequency component, and a non-periodic step maps to multiple frequencies.

of time-domain signals and their frequency-domain representations obtained with the DFT. The first row shows a constant signal, whose frequency representation consists of a single sinusoidal wave at 0 Hz. The second row shows a pure sinusoidal signal that is mapped by the DFT into a single frequency component. More complex signals, like the step function in the third row, include a larger number of frequency components.

The frequency-domain representation provides a description of signals according to their rate of change, or frequency content. The derivative of a signal is another signal that also describes its rate of change. This similarity can be seen as the intuitive reason why the frequency-domain description is convenient for analysing differential equations.

In the frequency-domain, systems (i.e., entities that take an input signal and generate an output signal) are described by how much they react to an input according to its frequency content, and more specifically by how much they amplify or reduce every frequency component of the input signal. This is measured by the ratio between the intensity of a given frequency component in the output and in the input. If the output over input ratio is smaller than 1, then we observe a reduction; conversely, if it is greater, then we observe amplification. When such ratio is equal to 1, we have unit amplification, which corresponds to a frequency component not being altered by the system. For example, many physical systems behave like a low-pass filter, transmitting or amplifying slow signals (i.e., low-frequency components), while reducing quickly changing components (i.e., high-frequency components). This behaviour of reducing a frequency component is called *filtering*.

In the case of control systems, we ideally want the physical quantity y to track the reference r at every time instant. In the frequency-domain this corresponds to unit amplification between the reference and the output at every frequency. This is apparently infeasible, since a change in r (a quantity in the cyber component that can be arbitrarily set) requires some time for y (a quantity of the physical world that has to obey the physics laws) to follow. In other words, reference changes that are too rapid cannot be tracked by the output of a system. In Figure 4, we show an example of how we could expect the drone to follow a step-like change in the desired position along one direction. The output y , denoted by the solid line, does not follow instantly the reference r , denoted by the dashed line. On the contrary, the quick change of reference value is smoothed in the output signal, which gradually reaches the new reference value. The frequency-domain concept of filtering can be used to describe this phenomenon. In fact, we could rephrase this as “the output only tracks the slowly changing (low-frequency) components of the reference, while it filters the fast-changing (high-frequency) ones.” We illustrate this intuitive interpretation in Figure 4 with red ellipses. The ellipse on the left highlights the filtering behaviour that occurs

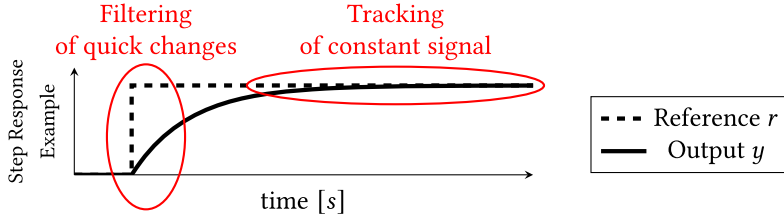


Fig. 4. Example of how we can expect the CPS output y (solid line) to track the desired value r (dashed line). In the figure, we intuitively highlight how a control system usually behaves like a low-pass filter, by filtering the fast-changing components of the input and tracking the slow-changing ones.

when the reference signal has a rapid change, while the one on the right highlights the tracking behaviour when the reference signal does not change.

Given this behaviour consisting of tracking the low frequencies and filtering the high frequencies, for a given control loop, we can identify the so-called *closed-loop bandwidth*, denoted by f_b . The closed-loop bandwidth is the threshold frequency below which we expect to have a tracking behaviour (i.e., $y \approx r$) and above which we have a filtering behaviour. This quantity therefore corresponds to the fastest frequency components of the reference that the control system is able to track. Accordingly, it is also considered a quantification of the speed of the control system: the higher the closed-loop bandwidth, the higher the rate of change of reference signals that it can track. We remark that such threshold, the closed-loop bandwidth, is a property of the system and should not depend on the specific input.

While it may seem intuitive that the control engineer wants to design a control algorithm that maximises f_b , to obtain tracking of fast-changing references, there are other factors to account for. As an example, a high value of f_b usually comes at the cost of a control actuation with a high value possibly leading to hardware damages, e.g., a fast-moving drone will generate high forces that can ruin the actuators. Furthermore, noise can be found in the measurements at high frequencies, e.g., an accelerometer that measures a drone's acceleration is usually affected by high-frequency electrical noise. Therefore, if the control system reacts to input signals in the high-frequency range (providing high speed), then it will also react to noise. In turn, this will reduce the system performance making its behaviour unpredictable. Such considerations lead to a trade-off in the control algorithm design between speed of the system and noise rejection.

3 CONTROL ENGINEERING PERSPECTIVE ON CPS STRESS TESTING

In this section, we first motivate and define the problem of stress testing the control layer in a CPS. Second, based on the development workflow of control-based CPSs illustrated in Section 2, we identify the types of assumptions that the engineers make in the different control software development stages. For each of these types, we discuss which techniques are already available for testing the corresponding assumptions and which assumptions require an application-specific solution; we exclude the latter from the scope of this work. For the remaining types, we discuss which assumption types should be tested first. We conclude this section by defining the problem addressed in this article.

3.1 Problem Motivation

As discussed in Section 2.2, when applying control theory, control engineers make *design assumptions* about both the physical part of the CPS and the control algorithm to be developed. The role of these assumptions is to abstract away the details of the system that are not necessary for developing control algorithms, and to define the fundamental building blocks (e.g., linear models)

used by the theory. For example, when designing the control algorithm of a drone, engineers assume that the generated thrust is proportional to the voltage command and that it does not saturate when the maximum power of the motors is reached. This allows engineers to use a linear model of how the voltage, when applied to the motors, affects the drone movement and position. This linearity assumption is necessary to apply traditional control theory [3].

As discussed in Section 2.1, software engineers are provided with the control algorithms from control engineers. These algorithms are only one component of the control layer. In fact, when implementing the latter, software engineers address the implementation of other functionalities such as the flight mode changes, the interaction with sensors and actuators (e.g., filtering and sanity checks), the parallel execution of the different control loops and the discretisation of the equations. The implementation and integration of other software functionalities can render the design assumptions made by control engineers invalid. For example, the linear model used for the control design is an assumption that is falsified when the drone motors saturate. This happens because the motors are requested to produce more thrust than their capacity, as in the cases when the reference value changes too much or too fast. In such scenarios, the drone control algorithm will be operating in conditions different from the ones assumed during design.

When testing the CPS implementation, software engineers have to be aware of possibly unpredictable software behaviour due to falsified design assumptions. For example, when testing the ability of a drone to perform a delivery, software engineers have to consider the possibility that the motors may saturate, thus leading to unpredictable behaviour. This implies that the flight performance of the drone might be impaired, potentially affecting the overall fulfilment of the requirements concerning the safe execution of the delivery. Conversely, software implementation choices can determine for which scenarios the design assumptions hold or not. For example, the update mechanism of the drone position reference values (e.g., based on external inputs or regular periods) can result in faster or slower reference changes, which can cause the saturation of the motors (and the consequent loss of guarantees that hold under design assumptions). Therefore, when generating test cases for the CPS control layer (as well as for the CPS overall) and evaluating the tests outcomes, software engineers have to consider the potential impact of falsified design assumptions.

Control algorithms are usually robust (at least to some degree) to the falsification of the different design assumptions; this property is one of the reasons for the successful adoption of control theory [3]. Control theory provides metrics to quantify the algorithm robustness to the deviation from assumptions, e.g., “stability margins”. However, those metrics are also based on the control design models and are still subject to the validity of the assumptions. Hence, *the quantification of the extent to which a CPS can be pushed outside of its design scope cannot be provided a priori by control theory and intrinsically requires empirical approaches or, in other words, testing*. This evaluation can be obtained through stress testing of the software that implements the control layer, by targeting the control design assumptions. Therefore, such a type of testing is ideally performed on the final implementation of the CPS. However, this is generally expensive in both time and resources. As a common alternative, implementation details (of both software and physical component) can be added to the simulation models and those can be used in place of the final implementation for testing their impact. The more exhaustive and adherent the additions are to the real implementation, the more the testing on the simulation model will be relevant to the CPS implementation.

3.2 Design Assumptions in Control Algorithms

Before defining our stress testing problem, we need to identify the types of design assumptions that control engineers make at design time. These assumptions are made at the different development stages of a control-based CPS. In Section 2.1, we identified three main development stages:

- (i) control problem definition,
- (ii) control algorithm design, and
- (iii) control algorithm implementation.

We now discuss the design assumptions made at each stage.

Assumptions at Control Problem Definition Time: At this stage the engineers identify the different control loops and modes for which they will develop a control algorithm. As a consequence, when designing the individual control loops, they assume that (i) the different loops do not interfere with each other and (ii) the mode changes do not impact the control design that follows [3]. For example, for a drone, the design of the altitude controller may not account for the horizontal controllers (and vice versa). Similarly, the design of the “aggressive flight” controllers is done independently from the “safe flight” controllers. Such assumptions significantly simplify the design of the control algorithms, allowing, among others, to independently design the response to a change in each element of the vector r . However, they do not always hold in practice. For example, when the drone tilts to move horizontally, it also loses vertical thrust, affecting the altitude controller. Another case of assumptions not holding is when a mode change command is issued during the flight. This can cause a sudden change in the motors’ commands, thus affecting the CPS performance.

Assumptions at Control Design Time: During the control design, the engineers develop a non-linear model of the physical part of the CPS, based in part on the information received from the physical-component engineers that designed it. Such a model (like any model) is only an approximation of reality and will not consider or only approximate certain aspects of the problem. For example, a drone model assumes a given mathematical relation between the rotational speed of the propellers and the generated vertical thrust. However, this type of aerodynamic phenomena is difficult to quantify. Moreover, there could be some inconsistency between the mathematical model and the real physical system. By using such models, the engineers implicitly assume that they are a sufficiently accurate representation of the physical reality.

As mentioned above, the models of the physics also need to be linearised to use the tools from control theory. The linearised version of the model is only valid in the surroundings of the operating point chosen for the linearisation. Practically, by using the linearised model, the engineers implicitly assume that, during operations, the CPS stays sufficiently close to the operating point so that the linearised model is an accurate enough representation of the physical part. For example, the propellers cannot generate more thrust than the motors can provide: the motors saturate (max-out) once they reach their maximum capacity. To linearise this relation, the engineers assume that the motors are not in the saturated state and that they always provide a thrust proportional to the voltage command. When, during the actual flight, the motors saturate, this proportional relation loses validity, as well as the model assumed during the control algorithm design.

Assumptions at Control Algorithm Implementation Time: Control algorithms are generally specified as linear differential equations. Such equations are defined with the use of continuous mathematics. However, they are implemented on computers, which are discrete machines. Hence, they have finite precision in the representation of the parameters and execute the algorithms in discrete steps over time. Accordingly, the engineers, when designing the control algorithm with continuous mathematics, are implicitly assuming that the discretisation happening during the implementation does not significantly alter the algorithm. More specifically, they assume both that the finite precision does not significantly alter the computed values, and also that the discrete execution does not alter the frequency properties (meaning the properties of the algorithm execution over time).

Design Assumptions Summary: To summarise, we identify the following types of design assumptions that are made by engineers during the development of control algorithms:

- A1** they ignore the interaction between different control loops;
- A2** they ignore the impact of mode changes on the control algorithms' performance;
- A3** they assume that the initial non-linear model of the physics is a sufficiently accurate representation of the real system;
- A4** they assume that the system stays sufficiently close to the operating point chosen for the linearisation so that the linearised model is valid;
- A5** they assume that the finite precision of the representation of the equation variables and parameters is adequate; and
- A6** they assume that the execution in discrete time steps does not significantly affect the expected execution time properties of the algorithm.

When performing stress testing for a control-based CPS, engineers should aim at falsifying each of these assumptions.

We note that there are branches of control engineering that aim at mitigating each of those simplifying assumptions, such as multivariable control (targeting **A1**) and robust control (targeting **A3**). However, like the stability margins mentioned above, such approaches are still subject to design assumptions and do not exclude the need for empirical verification. Furthermore, those are rather advanced theories and, as of now, find limited application in practice [18]. We now discuss which types of assumptions can be already stress tested with existing software testing techniques and which ones require an application-specific solution.

Testing numerical properties of numerical algorithms (Assumptions **A5** and **A6**) is not a novel problem; there is a significant literature corpus [25, 56], also targeting control algorithms [33, 48]. Similar considerations can be made about testing execution timing properties. A number of works can be found in the literature for testing real-time software [7, 32]. Furthermore, we note recent works dedicated to the verification and testing of the robustness of control algorithms to execution timing faults [21, 52]. Given the above previous works, we leave the testing of numerical and timing properties out of the scope of this article.

Testing the validity of the physical model (Assumption **A3**) is a highly application-specific problem. To test the aspects of the physics model that are unknown, one must know the aspects that were uncertain when developing it. For example, for a drone, two assumptions of the model can be on the aerodynamic properties of the propellers (needed to evaluate the vertical thrust that can be generated) and on the rigidity of the drone body (to simplify the equations describing the motion of the drone in space). Among those, the former is likely to be associated to a higher degree of uncertainty, because the aerodynamic phenomena are generally hard to characterise. In contrast, the assumption on the rigidity is more likely to be valid: intuitively, we do not expect the drone motor supports to bend. Such considerations are clearly application-specific and require an understanding of the specific model that is being used. Accordingly, the generation of test cases that falsifies this type of assumptions cannot be treated in a general fashion. Given its application-specific nature, we leave the testing of this type of assumptions out of the scope of this work.

We are therefore left with the assumptions regarding non-interactions between control loops (**A1**) and control modes (**A2**), and about the sufficiently large range of validity of the linear models (**A4**). Among those, we note that the first two depend on the latter. In fact, if an individual control loop does not have a sufficiently large range of validity when operating independently (i.e., without mode switches and in the absence of reference changes for the other loops), then, the switching across different modes and the interaction between loops are unlikely to improve its range. For example, if we have an altitude control loop for a drone that is not very robust when

operating alone, then it is unlikely to perform better when the control loops of the horizontal directions are also active and can disturb it. Given this dependency, we argue that *testing the validity of linear models should occur before testing the interactions between control modes and control loops*. In light of this discussion, this work focuses on the testing of the linearised model control design assumptions, for which we give our problem statement below.

3.3 Problem Statement

In this article, we address the problem of stress testing the implementation of a CPS control layer. In the control layer of a CPS there are several control loops; we focus on the problem of stress testing the linearised model design assumptions for the implementation of the individual loops. Our SUT is therefore an individual loop characterised by its reference value input and physical quantity output. We assume available basic information about the control-loop design and implementation, namely, an estimate of the closed-loop bandwidth from the control design and a range of valid values for the reference. The *objective* is to generate test cases that falsify the linearity design assumption, and identify when the falsification pushes the control algorithm out of its design scope (and hence causes the loss of the control-theoretical guarantees). To falsify the linearity assumption, we have to generate test cases where the physical component behaves differently from the (linear) model used during the control algorithm design. This difference in the behaviour should occur at various degrees and should increasingly make the control algorithm unable to provide the control-theoretical guarantees. The tests thus expose the robustness of the control algorithm to the different degrees of falsification of design assumptions. Accordingly, our *test inputs* exercise the control layer and consist of sequences of reference values over time. Our *test outputs* are the traces of the physical quantity that has to track the reference value.

To address the generation of stress test cases, we use control-domain knowledge to qualitatively characterise the input space of a control loop. To make the qualitative characterisation usable in practice, we propose a novel approach to the parametrisation of test cases for control-based CPSs based on the frequency-domain, in contrast to traditional time-domain approaches. Using such parametrisation, we can then introduce a novel metric to identify the stress test cases as well as different metamorphic relations that describe the expected behaviour patterns across different test cases. Leveraging the proposed parametrisation, metric, and metamorphic relations, we then develop a stress testing approach for the different control loops of a CPS control layer.

4 CONTROL LOOP INPUT SPACE CHARACTERISATION

In this section, we use domain knowledge from control theory to provide a *qualitative characterisation* of a single control loop input space. The proposed characterisation maps frequency and amplitude features of the input to the expected behaviour, i.e., the expected relation between the (scalar) output y of a control loop and the (scalar) input reference r . In the first part, we present the qualitative characterisation based on the validity boundaries of the linearised model and insights from control theory.⁴ We use a minimal example (a simplified model of the altitude control of a drone) to exemplify the different system behaviours highlighted by the characterisation. In the second part of this section, we list the qualitative aspects of the characterisation and propose approaches to quantify them. Such a quantification enables the practical use of the characterisation, and constitutes the basis for the test case generation approach proposed in the following section. We conclude the section by discussing our problem statement in the context of the proposed characterisation.

⁴We note that a similar qualitative characterisation of the input space of a control loop is found only in one book on control engineering from 1959 [22]. However, the treatment of the topic is brief and high-level and has not been investigated further in later literature.

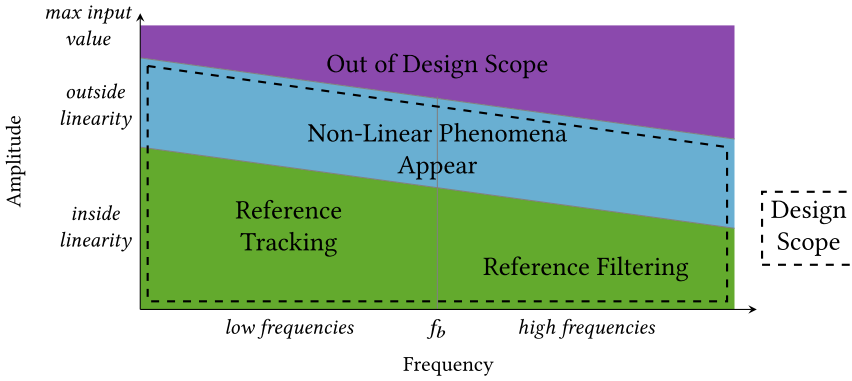


Fig. 5. Qualitative frequency-amplitude characterisation of the input space of a control loop. The colours highlight the validity of the linearised model with respect to the input frequency content and amplitude. The green area corresponds to the input signals for which the system remains within the assumptions of control theory. The azure area corresponds to the input signals that trigger non-linear phenomena but not enough to cause significant performance degradation in the system. The purple area corresponds to input signals for which control theory assumptions are not fulfilled and the behaviour of the system becomes unpredictable. We highlight the design scope (i.e., the union of the green and azure areas) with a dashed box.

4.1 Qualitative Input Space Characterisation

To leverage domain knowledge from control theory (Section 2.2), we base our characterisation on a frequency-domain description of the input sequences. This means describing the input space with two dimensions: one capturing the input frequency content and the second one capturing its amplitude. Being a two-dimensional space, the input space can be represented as a *frequency-amplitude plane*. We provide a graphical representation of this input space plane in Figure 5: one input sequence corresponds to one or more points on the plane according to its frequency content and its amplitude. We can use knowledge from control theory to identify different areas in the input plane according to the expected behaviour of the control loop, depicted by colours and boundaries in Figure 5. We identify these areas by checking where control-theoretical guarantees apply (i.e., checking the validity of the linear model), and where they exhibit a tracking and filtering behaviour within the applicability boundaries of control theory. In the figure, we draw the areas with *simplified* boundaries represented as straight lines (the dashed lines in the figure). Such lines are apparently a simplification and, in practice, the boundaries might not be represented as straight lines nor provide a clear division between the different areas. In the last part of this section (Section 4.3), we discuss in more detail the benefits and limitations of the proposed characterisation.

To exemplify the different behaviours highlighted in the characterisation, we use a minimal example of the altitude control of a drone. To enable the easy detection of the limitations of the linear model, we use a simulator based on a linear model and introduce one single source of non-linearity: the saturation of the thrust. This saturation limits the force that can be applied by the motors to move up and down the drone. In Figure 6, we report four executions showing the response to square waves with different amplitude and period values. For each execution, the top plot shows the desired altitude (the black line) and the actual altitude (the blue line). As discussed, the desired altitude is the input reference, and the actual altitude is the output physical quantity. The bottom plot shows instead the command sent to the motor that is used to accelerate or decelerate the drone (the red line). The saturation of the motor (and hence the validity of the linear

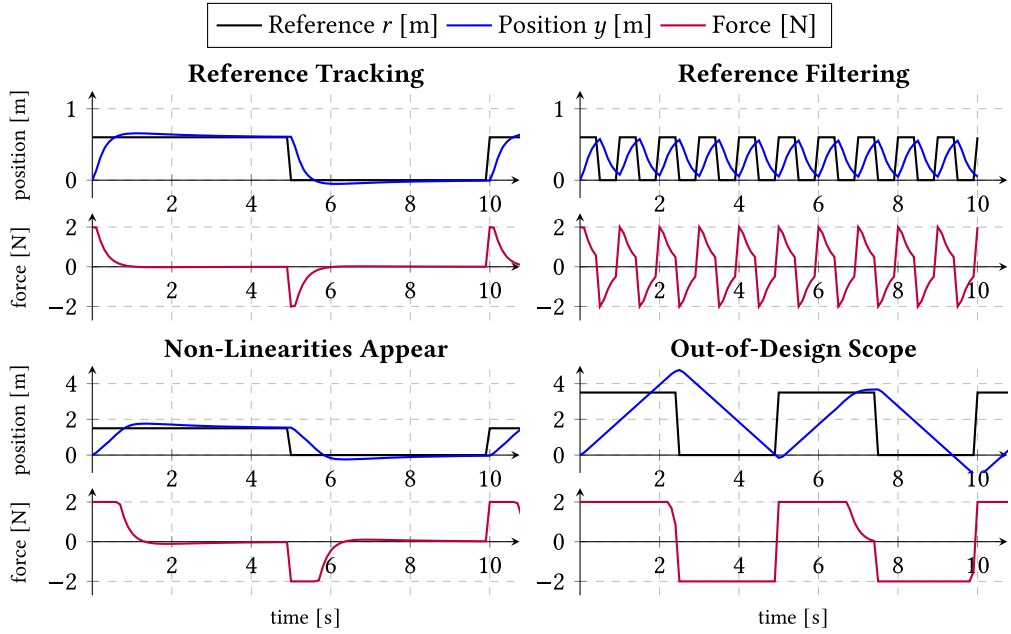


Fig. 6. This figure exemplifies the four different behaviours highlighted in the qualitative characterisation of the input space of a control loop. The plots are based on the simulation of a minimal model of the altitude control of a drone. The only source of non-linearity in this example is the saturation of the motor that limits (between ± 2 N) the thrust that can be generated by the motors. The blue lines show the altitude of the drone (i.e., the output y), the black line shows the desired altitude (i.e., the input r), and the red line in the bottom plots shows the thrust generated by the motors. Appearance of non-linear phenomena can be detected when the red line saturates at ± 2 N.

model) can be detected in these plots when the force becomes fixed at ± 2 N (see the bottom-left and bottom-right plots).

We first discuss the upper plots that concern flight simulations where motor saturation does not occur and that therefore are within the CPS design scope. We then discuss the bottom plots, where saturation occurs. Such tests are at the boundaries or outside of the CPS design scope. We conclude the description of Figure 5 discussing its connection with our definitions of design scope and stress tests.

Behaviours Within the Design Scope: Inputs consisting of small amplitude values will not push the CPS far away from its operational point. Accordingly, for lower amplitude values, we are within the validity bounds of the linear model: this is represented by the green area in Figure 5. Within this area, we expect the system to be able to track the slow-changing inputs: those sequences correspond to low-frequency inputs inside the “*Reference Tracking*” area. Fast-changing signals map instead to high-frequencies and are not expected to be tracked: those belong to the “*Reference Filtering*” area. In the figure, we highlight the closed-loop bandwidth f_b that separates the tracking and filtering behaviours, in the upper plots of Figure 6, we feed the controller of the drone with a slower (in the left-hand side plot) and a faster square wave (in the right-hand side plot), with an amplitude value equal to 0.6 m. In the former, we can see that the reference is successfully tracked within seconds after a step change. In the latter the reference changes are too fast and the drone cannot follow it successfully: We say that it is filtered.

Behaviours at the Boundary and Outside of the Design Scope: When we consider input signals that are larger in amplitude, the CPS moves further away from the operational point used for the linearisation and non-linear phenomena start to appear. In the altitude controller example this corresponds to hitting the motor saturation. Accordingly, in the bottom-left plot of Figure 6, we feed the drone with a larger square wave with an amplitude equal to 1.5 m. As we can see from the plot of the control action, the motor saturates for some time after the occurrence of the step in the reference. This, however, does not significantly affect the way that the actual altitude of the drone follows the desired reference, i.e., the reference is still successfully tracked. Since the reference is successfully tracked, we can consider this test to be still within the design scope of the CPS, despite the occurrence of motor saturation. In other words, the control algorithm is showing some robustness to the motors being saturated over a limited amount of time. Accordingly, in Figure 5, such signals correspond to the azure area “*Non-linear Phenomena Appear*,” which we consider part of the design scope as the CPS behaviour is not impaired. When the system moves even further away from the design scope, the linearised models are falsified even more and there is no way to predict the system behaviour. This is the “*Out-of-design Scope*” purple area. For example, in the bottom-right plot of Figure 6, we can see that the drone is not only unable to track the square wave with an amplitude value of 3.5 m, but it also exhibits a new behaviour, a triangular wave.

Finally, we note that either large (high amplitude) or fast-changing (high frequency) inputs can lead the system out of its design scope. For example, in the drone altitude control, either a fast-changing input or a large input can require high thrust and hence can cause motors saturation. Furthermore, fast change combined with a large inputs lead to a compounded effect on the validity of the linear model. Accordingly, in Figure 5, we depict the thresholds for which non-linear behaviours appear and the bound of the design scope (i.e., the green area) as decreasing when frequencies increase.

To summarise, given the qualitative plot of Figure 5, the design scope is identified as the union of the green and azure areas; we highlight this area using a dashed box. The system is instead considered out of the design scope when the occurrence of non-linear phenomena affects the CPS behaviour, which corresponds to the purple area. Accordingly, the stress tests are the tests that cover the azure area and especially its boundary with the purple area. Those are the tests where the non-linear phenomena appear and become large enough to affect the CPS behaviour, thus requiring empirical verification.

4.2 Qualitative Aspects of the Characterisation and their Quantification

We aim to use the qualitative characterisation proposed in the previous section to generate test cases that falsify the linearised model used for the algorithm design, and hence push the control layer to its performance limits. In other words, we want to sample (test) points in the frequency-amplitude plane and identify the behaviour that the test results expose in various areas of the plane. To stress test the CPS, we want to sample around the border of the “out-of-scope” area to understand when the control algorithm is no longer able to provide the performance guarantees. Furthermore, we also want to identify the border between “tracking” and “filtering” behaviours to characterise the fastest signals that the control algorithm can track.

We note that the latter distinction between the “tracking” and “filtering” areas (i.e., the closed-loop bandwidth) is not related to the falsification of the design assumptions (both areas are in fact coloured green). However, it represents how fast the control loop can track a reference and hence represents a performance limit of the system. If we want to push the system to its performance limits, then we have to ensure that the test cases cover both behaviours.

Accordingly, to make our qualitative characterisation usable for the generation of stress test cases, we have to make quantitative the following qualitative aspects:

- One input sequence generally contains more than one frequency, and hence can map to more than one point in the frequency-amplitude plot. Accordingly, we need to define a *mapping between a test input sequence to a corresponding set of frequency-amplitude coordinates* in the input plane. This enables the identification of which points in the frequency-amplitude plane are sampled by a test (Section 4.2.1).
- Detecting when a test trace shows a behaviour altered by the falsification of the linearised model—when a test belongs to the purple area—does not have a formal definition in the existing literature. Accordingly, we need to define a “*degree of non-linearity*” observed in a given test result. Such a degree of non-linearity should capture the impact of non-linear phenomena on the CPS behaviour. In other words, we want to detect test cases that belong to the purple area and that are outside of the scope of the design assumptions (Section 4.2.2).
- Since a test can map to multiple frequency-amplitude points, it can expose multiple behaviours simultaneously. Accordingly, we need to define a *mapping between the different behaviours observed in a given test and its frequency-amplitude points* (i.e., the different coordinates mentioned above). This enables the distinction of the different behaviours (tracking, filtering, out of scope) that might appear in the same test (Section 4.2.3).
- It is practically impossible to know *a priori* (1) the actual shape of the threshold for which non-linear phenomena start to appear, and (2) the threshold for which they falsify the linear models to a sufficient degree to impair the system performance. Hence, the bounds between the areas of different colours can have an arbitrary shape. However, we can *use the relative positioning of the behaviour areas to define a set of properties that are expected to hold when comparing the behaviours exposed by test cases with different frequency and amplitude content* (e.g., filtering will appear at higher frequencies than tracking). This enables the definition of test case generation strategies in the frequency-amplitude plane that explore the different behaviours of the control loop (Section 4.2.4).

We now address the definition and quantification of each of these qualitative aspects. Since we leverage the DFT of the input and output of the tests, in Figure 7, we show the frequency-domain representation for the tests of Figure 6. The figure uses the same colour convention as its time-domain equivalent: blue crosses represent the frequency components of the trace of the actual position of the drone, and black crosses represent the frequency components of the input sequence. We use the plots of the DFT to exemplify the different definitions and explain the underlying intuitions. We remark that, based on common practice in the frequency-domain, we use a logarithmic scale on both the axes of all the plots in Figure 7. This enhances the readability of the plots.

4.2.1 Mapping of Tests to Frequency-amplitude Points. Given an input reference sequence $r(t)$, we want to define the frequency-amplitude coordinates sampled with the associated test. We define this mapping according to the frequency spectrum (the DFT) of the input reference. In practice, inputs are signals sampled over time: therefore, the spectrum computed with the DFT is also discrete [16]. More specifically, the time-domain samples are mapped to an equal number of evenly spaced frequency components, like in the DFT examples in Figure 3. The number of the frequency components is therefore large: for example, a 5 seconds trace sampled every millisecond is mapped to 5000 frequency components. However, most of those components are usually equal to zero or close to it, meaning that only few of the frequency samples actually carry information about the signal.⁵ Accordingly, among all of the frequency components computed with the DFT, we consider

⁵The reason for such excess of samples is that signals are usually oversampled (sampled more frequently than strictly necessary) for redundancy and robustness. This oversampling introduces extra frequency components in the higher part of the spectrum that do not carry much information about the signal and are therefore zero or close to it.

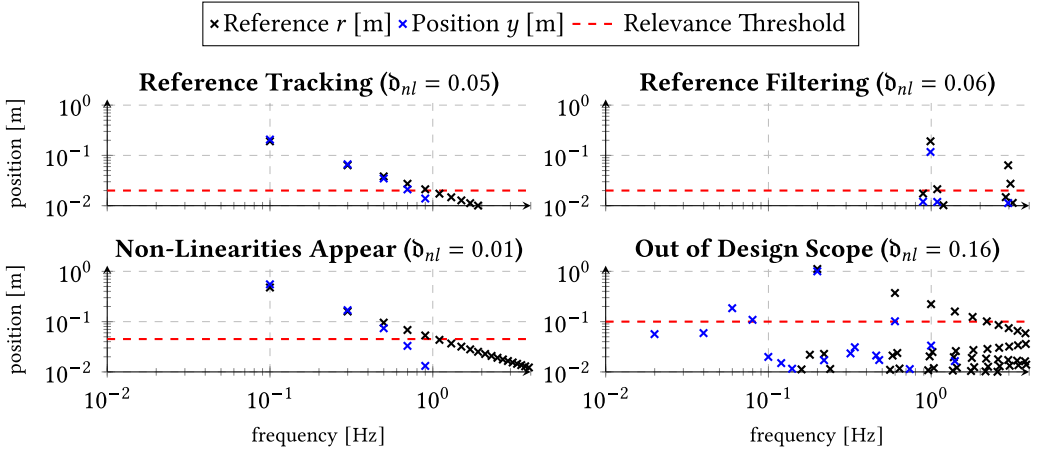


Fig. 7. DFT of the input (black crosses) and output (blue crosses) traces from the tests on the minimal example of the drone altitude control shown in Figure 6. These plots exemplify how we use the frequency content of the input and output to detect the different behaviours of control systems. The filtering behaviour is detected when input components are not found in the output (top-right plot). Non-linear behaviour is detected when new frequency components are found in the output (bottom-right plot). Furthermore, the red dashed line highlights the threshold that we adopt to identify the relevant input components used for the mapping of the test case to the frequency-amplitude plane.

as relevant only the ones with larger amplitudes. Formally, given an arbitrary input $r(t)$, we map it to a set of frequency-amplitude coordinates $\langle f, A \rangle$:

$$fAmap[r(t)] = \{ \langle f, A \rangle : A = |DFT[r(t)](f)| \wedge A > \rho \cdot \max_f \{|DFT[r(t)](f)|\} \}, \quad (1)$$

where $DFT[\cdot]$ denotes the DFT, $|\cdot|$ denotes the modulus, and ρ is a parameter in the range $[0, 1]$ that we use to select the relevant components (i.e., the larger ones) in a relative way to the largest one: $\max_f \{|DFT[r(t)](f)|\}$. We exemplify the definition of this relative threshold (used for selecting the relevant input components) with the red dashed line (for $\rho = 0.1$) in the plots of Figure 7. We can then observe in Figure 7 that: the faster square waves of the two right-hand side plots map to points further to the right on the frequency axis than the other tests associated with slower square waves. Furthermore, the larger amplitude values of the square waves from the bottom plots map to points higher in the amplitude axis than the other tests associated with smaller square waves. This exemplifies how the size and the speed of the inputs are captured in the frequency-domain.

4.2.2 Degree of Non-linearity Definition. With the degree of non-linearity, we want to detect when non-linear phenomena impact the CPS behaviour. As exemplified in the bottom-right plot of Figure 6, when non-linear phenomena push the CPS out of its design scope, they introduce unexpected behaviours in the output. This kind of behaviour can be harmful as it implies that the control algorithm is introducing some new behaviour in the system that was not part of the reference. For example, in the bottom-right plot of Figure 6 the altitude of the drone reaches 4.5 m when the reference is at 3.5 m. The bottom-right plot of Figure 7 shows the DFT of the input and output of the test exhibiting non-linear behaviour in our drone altitude control example. In this example, we remark the presence in the output (the blue crosses) of components in the frequency range $[0.02, 0.4]$ Hz that were not present in the input (the black crosses).

According to this intuition, we define the degree of non-linearity on the base of new output frequency components that appear outside of the input spectrum. This requires *sampling the output in*

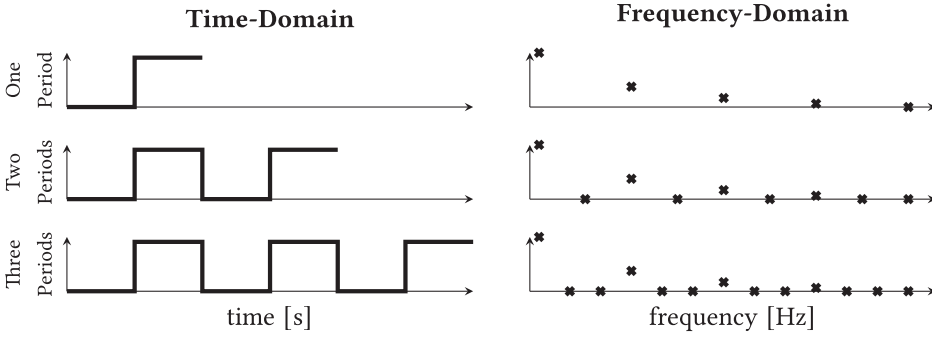


Fig. 8. Graphical representation of how the repetition of the same input sequence increases the resolution in the frequency-domain. The repetition of the same sequence does not introduce new information to the input signal, whose non-zero frequency components remain unchanged. However, when the input is repeated, the output signal is also sampled in frequencies outside of the main input components (the ones that have a value equal to zero in this plot) hence enabling the detection of frequency spectrum broadening because of non-linear behaviour.

between the frequencies of the input components. To increase the number of samples in the frequency spectrum without altering the frequency content of the input, we can repeat the input sequence (i.e., make it periodic). This does not alter the information contained in the input (since it is just repeated), and the non-zero frequency components do not change. However, for each repetition, we double the samples in the time-domain, resulting in doubling the samples in the frequency-domain. The new samples obtained in this way are found on the frequency axis *between* the samples previously available (rather than only in the higher part of the spectrum, as for the samples introduced by the oversampling). We exemplify graphically the sampling of new frequencies in Figure 8. The figure shows how repeating the steps in the time-domain (plots on the left-hand side) increases the number of samples in the frequency-domain (plots on the right-hand side) by adding new zero-valued frequency samples in between existing samples, hence increasing the sampling resolution.

When identifying the impact of non-linear phenomena on the CPS, we are interested in detecting *frequency components that were small in the input and become large in the output*. Hence, we define the degree of non-linearity according to the *maximum amplitude* of the output spectrum outside of the relevant components of the input spectrum. We look therefore at all the frequencies sampled in the DFT of the output y , excluding the relevant ones found in the input (i.e., $fAmap[r(t)]$). Formally, given a reference sequence $r(t)$, we can define the set of frequencies to check as $f_{new} = \{f : \exists DFT[r(t)](f) \wedge \neg \exists (\langle f, * \rangle \in fAmap[r(t)])\}$, where $*$ can be any amplitude value. For example, in the tests of Figure 6 it means that we are looking at the frequencies that are *not* associated with any input frequency component (black cross) above the red dashed line. Such an amplification of frequency components only appears in the bottom-right plot, where the saturation time is sufficiently long to affect the CPS behaviour. Differently, in the bottom-left plot, saturation occurs but not for long enough to alter the CPS behaviour. Furthermore, to obtain comparable results across tests with different amplitudes, we normalise our metric with respect to the amplitude of the input $\max_f \{|DFT[r(t)](f)|\}$ (remarking that the sampled frequencies are necessarily the same for input and output across the tests). The intuition is that a deviation from a large reference change matters less than the same deviation from a small reference change. We then obtain the following definition for the **degree of non-linearity** of a given test i :

$$d_{nl}(i) = \frac{\max_{f \in f_{new}} \{|DFT[y_i(t)](f)|\}}{\max_f \{|DFT[r_i(t)](f)|\}}, \quad (2)$$

where $r_i(t)$ and $y_i(t)$ are, respectively, the input and output associated with the test and the other elements follow the same conventions as in previous equations. When we apply the $\mathfrak{d}_{nl}$ formula to the examples in Figure 7, we obtain the values reported in the titles of the different plots.⁶ By comparing the numbers, we see that the new frequencies that appear in the bottom-right plot cause the $\mathfrak{d}_{nl}$ to be one order of magnitude higher than the $\mathfrak{d}_{nl}$ of the other tests. This example also shows that having a new frequency component with 16% of the maximum input amplitude (caused by non-linear phenomena) can significantly impact the system's performance.

We exemplify the use of this metric with our examples in Figure 7. As indicated in the plots titles, for the tests where the saturation is not triggered, we measured a $\mathfrak{d}_{nl}$ of 0.05 and 0.06. In the bottom-right test that shows misbehaviour, we desirably obtain a higher value of 0.16 for our metric. Finally, in the bottom-left plot, we measure a $\mathfrak{d}_{nl}$ value of 0.01, hence lower than the one in the upper plots. Such a low value might seem counter-intuitive given that in this test the saturation is triggered but, thanks to the algorithm robustness, in this case the saturation does not alter the behaviour of the CPS by introducing new potentially dangerous components in the output. Therefore, such a low $\mathfrak{d}_{nl}$ value allows us to distinguish when the occurrence of non-linear phenomena, actually pushes the CPS out of its design scope (bottom-right test) from when it does not (bottom-left test). Instead, the fact that the $\mathfrak{d}_{nl}$ value is even lower than the value of the upper plots can be attributed to numerical noise. Indeed, in each of the three tests, no large amplitude is found in the f_{new} set of frequencies: amplitude values are small or close to zero and small variations caused by numerical noise can alter the values. However, when non-linear behaviour appears and larger amplitudes are found in f_{new} , the numerical noise becomes less relevant.

To conclude, we note that the idea of repeating the input sequence comes with a trade-off between the test duration and the frequency resolution. Higher frequency resolution increases the chances of detecting new frequency components, hence non-linear behaviours; however, more repetitions require longer tests. The number of frequency samples needed to detect new frequencies in the output depends on the specific application. Accordingly, such a number can be evaluated empirically using a manual test, with several input repetitions, that shows non-linear behaviour (this test can be obtained using high amplitude and frequency content, and an arbitrary shape). Then, by computing the $\mathfrak{d}_{nl}$ using different trace lengths (corresponding to different numbers of repetitions), we can evaluate how many periods are needed so that the frequency samples are sufficient to detect new undesired output components.⁷

4.2.3 Mapping of Behaviour to Frequency-amplitude Points. When we analyse the degree of non-linearity, we obtain a metric that characterises all the input frequency-amplitude components. In fact, by looking at the $\mathfrak{d}_{nl}$ metric, there is no way to identify which input component causes the non-linear behaviour.

Differently, when the SUT behaves linearly, some frequency components of the same input are tracked and pass through to the output, while other components are filtered. For example, in the top-right plot of Figure 7, we can observe that the input component at frequency 1 Hz is found, albeit reduced in amplitude, also in the output, while the component at frequency 3 Hz has a much lower amplitude value in the output. Therefore, when we quantify the filtering behaviour of tests exposing linear behaviour, it is necessary to analyse the different frequency components individually.

For each of the frequency-amplitude component of the input, we define a *degree of filtering* based on the ratio of the output and input amplitudes. If a frequency component is perfectly tracked, then

⁶In this example, we used ten repetitions of the step sequence to compute the $\mathfrak{d}_{nl}$.

⁷We exemplify the procedure of selecting the number of input repetitions in the experimental part of this work, specifically in Section 6.2.

its amplitude in the input and output are equal, hence yielding a ratio equal to 1. Instead, if the ratio of the input over the output is below the unit, then it corresponds to filtering, since part of the signal is lost. Analogously, values above 1 indicate amplification: while a small amplification can be expected in real-world systems, large amplification can be dangerous (for the very same reason as the risks of broadening the frequency spectrum).

Hence, given a test i , we define the degree of filtering for a given input frequency component f as the difference between 1 and the mentioned output-input ratio:

$$\mathfrak{d}_f(i, f) = 1 - \frac{|DFT[y_i(t)](f)|}{|DFT[r_i(t)](f)|}, \quad (3)$$

with the same conventions as for the equations above, and the caveat that this definition is valid only for tests that expose linear behaviour. Given the absolute values at the numerator and denominator, this metric takes values in the range $[-\infty, 1]$. A value of 1 describes complete filtering, while 0 describes perfect tracking. Negative values correspond to input amplification.

4.2.4 Expected Properties of the Characterisation. Due to the qualitative nature of the characterisation presented in Figure 5, we cannot use it to predict the exact shape of the different behaviour areas. However, we can use the relative positioning of the areas to define properties over the tests. More specifically, the areas' relative positioning shows that (1) tests with higher frequency content and larger amplitudes are expected to push the CPS further away from the operation point used for the linearisation and cause non-linear behaviour, and (2) tests with higher frequency content are expected to expose more filtering behaviour than ones at lower frequencies.

We therefore formulate the following expected properties⁸:

- PR1** The degree of non-linearity should increase when the amplitude and frequency values increase. In fact, the further we move away from the origin of the frequency-amplitude plane, the closer we should be to the input area outside of the design scope.
- PR2** For tests within the design scope (i.e., tests that show linear behaviour), the filtering degree should increase as the frequency increases. In other words, faster signals should always be harder to track than slower ones.
- PR3** The closed-loop frequency bandwidth should be independent of the specific test. In fact, when the system behaves linearly, the threshold between the tracking and filtering areas should not depend on the specific input and be instead a property of the system [3].

We now exemplify an evaluation of these properties on the tests of Figures 6 and 7. Property **PR1** is fulfilled by all tests except for the bottom-left one. In fact, the test with the highest amplitude (the bottom-right one) is the one with the largest $\mathfrak{d}_{nl}$ and the one with the second highest $\mathfrak{d}_{nl}$ is the one with the fastest input (the upper-right one). However, when we compare the two plots on the left-hand side, we would expect the bottom one to have a higher $\mathfrak{d}_{nl}$ than the upper one, since the latter receives an input with the same frequencies but smaller amplitude. The exception of the bottom-left plot showcases that, when the linear model loses validity (in this case because of saturation), the behaviour of the system becomes unpredictable, though not necessarily worse as illustrated by the low $\mathfrak{d}_{nl}$ value. This unpredictability further underlines the importance of testing in the areas of the input space that are close to the design assumptions validity bounds. Property **PR2** is fulfilled: looking at the tests exhibiting linear behaviour, we can observe that the blue crosses move further down in the position axis from the black ones as we move to higher frequencies. This means that the reference signal is found increasingly less often in the output. Concerning Property **PR3**, we can observe, in the tests shown in the top part of the figure, that

⁸In Section 5.2, we formalise the properties into metamorphic relations.

the frequency above which the input is filtered in the output (i.e., the closed-loop bandwidth) is similar for both tests (around 0.9 Hz). In other words, the f_b measured from the two tests in the upper plots is similar and thus the two tests comply with the third property. For the bottom-left test, the frequency at which the input is filtered is lower (around 0.6 Hz). This is possibly due to the appearance of the saturation that limits how large and fast references the drone can track (hence practically decreasing the closed-loop bandwidth for larger amplitudes). Given that in the real world we would not necessarily know that the bottom-left test is triggering some non-linear phenomenon, this discrepancy from the two upper tests can be used to highlight that this test might require a more detailed analysis (even though the control performance is possibly still acceptable).

4.3 Benefits and Limitations of the Frequency-amplitude Characterisation

Our testing objective is to generate and identify stress test cases that push the system around the limits of validity of the linearised model. In test cases where non-linear phenomena appear, the control-theoretical guarantees are gradually lost (depending on how far the behaviour of the CPS deviates from the one described by the model) and new unpredictable frequency components appear in the output. Accordingly, we propose the use of the $\mathfrak{d}_{nl}$ metric, as a measure of the new frequency components that appear in the CPS output, to identify the stress test cases and therefore whether a test is inside or outside of the design scope. In other words, $\mathfrak{d}_{nl}$ captures and quantifies non-linear behaviour. To specifically explore the *validity limits* of the design assumptions, we are interested in test cases where the $\mathfrak{d}_{nl}$ is non-zero (hence not being fully within the design scope) but also not too large (hence not being far outside of the design scope).

However, the identification of a boundary in the input space where the $\mathfrak{d}_{nl}$ transitions from zero to non-zero values is not possible for arbitrary input sequences. In fact, as discussed above, the $\mathfrak{d}_{nl}$ metric equally characterises all of the frequency-amplitude components of a given test. Consequently, a given point in the frequency-amplitude plane can lead to different $\mathfrak{d}_{nl}$ values depending on the other components of the input. For example, in both the two right-hand side tests presented in Figure 7, we can identify a main component around 1 Hz and amplitude 0.2 m. The two components are located very close in the plane but are associated with two tests that yield different $\mathfrak{d}_{nl}$ values (and accordingly different behaviours). The test depicted in the upper plot shows linear behaviour and is within the design scope, while the test in the bottom plot is outside of the scope. This discrepancy appears because the two components belong to different tests and are therefore coupled with different other components. This further highlights that Figure 5 is a simplified representation of the input space with clear-cut boundaries. Depending on the other frequency-amplitude components that one point in the frequency-amplitude plane is paired with in a given test, it can map to different behaviours (i.e., $\mathfrak{d}_{nl}$ values). The exposed behaviour of a point in the plane can in fact change according to which other frequency components are present in the same input.

To overcome the above limitation, in our testing approach, we propose a novel test-case parametrisation for the control layer of CPSs. With this parametrisation, we separate, in a test case, the definition of

- the frequency content,
- the amplitude content, and
- the combination (i.e., the relative positioning) of the different frequency-amplitude components.

As shown in the next section, thanks to this separation, we can leverage the proposed characterisation to control the expected behaviour of the tests.

5 TESTING APPROACH

In the first part of this section, we propose a novel test case parametrisation for the control layer of CPSs. The objective of the proposed parametrisation is to enable the definition of test cases according to the concepts presented in the previous section. More specifically, we design our parametrisation with the following objectives:

- it allows the use of the degree of non-linearity $\mathfrak{d}_{nl}$ (Equation (2)),
- it distinguishes possible choices for the different input features (frequency content, amplitude content, and the combination of different frequency-amplitude components), and
- it enables the definition of the expected properties from Section 4.2.4 as metamorphic relations [4, 14].

In the second part of this section, we define the mentioned metamorphic relations. In the third part, we use our test case parametrisation and the metamorphic relations to develop our testing approach. We use the $\mathfrak{d}_{nl}$ and the expected property **PR1** to obtain a preliminary optimistic bound on the input amplitudes that the control layer can track. Using the preliminary bound, we use our heuristic sampling method to generate test cases that explore the area of the frequency-amplitude plane where stress test cases are likely to appear. We then execute the test cases, and use the $\mathfrak{d}_{nl}$ and the $\mathfrak{d}_f$ metrics, together with the metamorphic relations, to identify test cases outside of the design scope.

5.1 Test Case Parametrisation

One test case i corresponds to a sequence $r_i(t)$ of reference values over time t . To achieve the parametrisation objectives listed above, we define $r_i(t)$ as a function defined by three elements: an amplitude gain, a time-scaling coefficient, and a periodic shape function. More specifically, we use the expression

$$r_i(t) = \alpha_i \phi_i(\gamma_i t), \quad (4)$$

where:

$\phi(\cdot)$ is a periodic function defining the input shape. For example, it can define a square wave or a triangular wave (arbitrary sequences can be chosen as long as they are periodic). Without loss of generality, we assume that shape functions are normalized to have a unit period (i.e., $\forall t : \phi(t) = \phi(t + 1)$) and also have a unit amplitude range (i.e., $\max_t \phi(t) - \min_t \phi(t) = 1$).

α is a gain used to scale the input amplitude.

γ is a time-scaling coefficient that changes how quickly we go through the input shape.

One test case i is therefore fully defined by a triplet $\langle \phi_i, \alpha_i, \gamma_i \rangle$. We note that, as long as ϕ is a periodic function, the sequence $r_i(t)$ is also periodic. Hence, it is a repeated sequence and enables the use of the proposed $\mathfrak{d}_{nl}$ (Equation (2)).

Sampling of Frequency-amplitude Coordinates: We now use the reference square waves from the examples in Figures 6 and 7 to exemplify how the proposed parametrisation works and how it enables intuitive sampling of the frequency-amplitude plane. More specifically, we describe how the parametrisation separates the choice of the relative positioning of the different main frequency-amplitude components and of their frequency and amplitude values. The four tests from the figures can be defined using our parametrisation with the ϕ function being a square wave with unit period, and the reference position switching between 0 and 1. The α value corresponds to the amplitude of the square wave and, hence, the tests in the upper plots both have an amplitude gain of 0.6, while the bottom plots have an amplitude gain of 1.5 and 3.5, respectively. Concerning the values of γ , the left-hand side plots' period is 10 and hence have a time scaling of 0.1 to make the signal

slower. The right-hand side plots have instead periods equal to 1 and 5, respectively, and hence corresponding time-scaling values of 1 and 0.2.

The shape function ϕ defines the relative positioning of the different $\langle f, A \rangle$ test coordinates. In other words, it defines their pattern (e.g., combination of larger and smaller components) in the frequency-amplitude plane. However, thanks to the linearity of the DFT, this pattern is independent of the α and γ coefficients. For example, if we look at the main frequency-amplitude components of the inputs in Figure 7 (the black crosses above the red dashed line), then we can see that the different square waves all map to components along a decreasing straight line.⁹ This shows that the relative positioning of the main components is defined only by the shape function and is independent of the scaling coefficients α and γ .

The α coefficient enables the movement of the $\langle f, A \rangle$ test coordinates along the amplitude axis. Increasing (decreasing) its value makes the frequency-amplitude components map to components further up (down) in the plane. Thanks to the linearity of the DFT, multiplying the value of α proportionally increases the main components' amplitude coordinate A [16]. For example, comparing the two left-hand side plots of Figures 6 and 7, we can see that the two square waves with the same period map to components with the same frequency but scaled amplitude.

The γ coefficient enables the movement of the $\langle f, A \rangle$ test coordinates along the frequency axis: i.e., the choice of the horizontal coordinate f . Analogously to the amplitude, increasing (decreasing) its value increases (decreases) the speed with which we go through the shape function and thus its frequency content. Due again the linearity of the DFT, a scaling of this coefficient corresponds to an equivalent scaling of the main components' f coordinates. For example, comparing the two upper plots of Figures 6 and 7, we can see that the two square waves that have the same amplitude but different periods map to points with the same amplitude but, respectively, lower and higher frequency.

5.2 Definition of Metamorphic Relations

The properties introduced at the end of Section 4.2 describe relations between different tests. For example, we used such properties to describe the relation between the four tests in Figures 6 and 7. Properties that concern the inputs and outputs of multiple test cases are known as **metamorphic relations (MR)** [14]. We define one MR for each property. We define the first two MRs as implications over tests with the same shape. Accordingly, each one contains a *condition* and an expected *implication*. The conditions concern relations between the amplitude and frequency content of different test inputs as well as whether the output shows non-linear behaviour or not. The implications concern relations between the $\mathfrak{d}_{nl}$ and $\mathfrak{d}_f$ observed in the tests outputs. In contrast, we define the third MR as a property across tests with different shapes.

Using our test case parametrisation, for tests based on the same shape, we can directly identify relations in the input amplitude and frequency content using parameters α and γ . More specifically, given two tests i and j with the same shape (i.e., $\phi_i = \phi_j$), the relation between α_i and α_j (γ_i and γ_j) identifies their relation in terms of amplitude (frequency) content. For example, $\alpha_i > \alpha_j$ implies that test i maps to points higher in the frequency-amplitude plane with respect to points for test j . Analogously, $\gamma_i > \gamma_j$ implies that test i maps to points further to the right in the frequency-amplitude plane with respect to points for test j .

Property **PR1** states that an increase in amplitude or frequency content of the input should cause an increase in $\mathfrak{d}_{nl}$. Thanks to our characterisation, for two tests i and j based on the same shape ($\phi_i = \phi_j$), an increase in amplitude and frequency are identified with $\alpha_i > \alpha_j$ and $\gamma_i > \gamma_j$.

⁹Square waves map to a decreasing straight line in logarithmic scale. In linear scale they map to a hyperbole, as exemplified in Figure 8.

Their conjunction can be used to define the MR condition. The MR implication instead concerns an increase in the degree of non-linearity. This can be expressed as $\mathfrak{d}_{nl}(i) > \mathfrak{d}_{nl}(j)$. Hence, we can write the following MR:

$$\text{MR1: } (\phi_i = \phi_j) \wedge ((\alpha_i > \alpha_j) \wedge (\gamma_i \geq \gamma_j) \vee (\alpha_i \geq \alpha_j) \wedge (\gamma_i > \gamma_j)) \implies \mathfrak{d}_{nl}(i) > \mathfrak{d}_{nl}(j). \quad (5)$$

As described in Section 4.2.4, this MR is, for example, satisfied by the upper-left and bottom-right tests of Figure 7, where the increase of γ from 0.1 to 0.2 and the increase of α from 0.6 to 3.5 lead to an increase in $\mathfrak{d}_{nl}$ from 0.05 to 0.16. Differently, it is not satisfied by the tests of the left-hand side plots, where an increase of α from 0.6 to 1.5 and same γ correspond to a decrease in $\mathfrak{d}_{nl}$ from 0.05 to 0.01.

Property PR2 states that, as long as the tests show linear behaviour, an increase in the frequency content will correspond to an increase in the filtering behaviour. To identify linear tests, we use a threshold t_{nl} on our degree of non-linearity. Tests below such threshold ($\mathfrak{d}_{nl}(i) < t_{nl}$) are therefore considered to show linear behaviour, and we limit the definition of this MR to those. Like for the previous MR, we identify an increase in the frequency content for two tests with the same shape as a greater time-scaling parameter $\gamma_i > \gamma_j$. The conjunction of the $\mathfrak{d}_{nl}$ threshold and the increase in frequency content constitute the MR condition. For the MR implication, an increase in the filtering behaviour can be identified with an increase of the $\mathfrak{d}_f$. However, different from the $\mathfrak{d}_{nl}$, the $\mathfrak{d}_f$ applies to each main frequency-amplitude component of the test. Hence, we need to evaluate it for each component of the inputs, i.e., $\forall f | \langle f, A \rangle \in fAmap[r_i(t)]$, and compare it with the corresponding component of the other test, i.e., the one with frequency $f \cdot \frac{\gamma_i}{\gamma_j}$ (leveraging the linearity of the DFT, the proportion in the γ coefficients matches the proportion in the frequencies). Using this quantification of the implied increase in $\mathfrak{d}_f$, we obtain

$$\text{MR2: } (\phi_i = \phi_j) \wedge (\gamma_i > \gamma_j) \wedge (\mathfrak{d}_{nl}(i) < t_{nl}) \wedge (\mathfrak{d}_{nl}(j) < t_{nl}) \implies \forall f | \langle f, A \rangle \in fAmap[r_i(t)], \mathfrak{d}_f(i, f) > \mathfrak{d}_f(j, f \cdot \gamma_j / \gamma_i). \quad (6)$$

For example, this MR is satisfied by the upper tests of Figure 7, where the increase of γ from 0.1 to 1 corresponds to an increase in $\mathfrak{d}_f$ for the main frequency component (respectively, the ones at 0.1 and 1 Hz). In fact, the blue cross (representing the output frequency component) moves further down along the amplitude axis with respect to the input component, hence highlighting filtering action. More precisely, the $\mathfrak{d}_f$ value of the main component of the upper-left plot is 0 at 0.1 Hz and the $\mathfrak{d}_f$ value of the upper-right plot is 0.45 at 1 Hz.

Property PR3 states that, as long as the tests show linear behaviour, the closed-loop bandwidth should not depend on any specific test. We can use the $\mathfrak{d}_f$ metric to identify the closed-loop bandwidth. According to its control-theoretical definition, the closed-loop bandwidth f_b corresponds to a ratio of 0.5 between output and input [3]. Therefore, the bandwidth can be identified as the threshold below (above) which the frequency components show a $\mathfrak{d}_f$ lower (higher) than 0.5, determining the reference tracking (filtering) area. To complement the two previous MRs that concern tests having the same shape, we use this property to define an MR across different shapes. We denote the closed-loop bandwidth estimated with the tests of a given shape ϕ as the threshold $f_{b,\phi}$. Therefore the third MR can be defined as the expectation that the f_b estimated using two tests i and j with different shapes are similar

$$\text{MR3: } |f_{b,\phi_i} - f_{b,\phi_j}| < \epsilon, \quad (7)$$

where ϵ is a small discrepancy that can be accepted. As described in Section 4.2.4 when exemplifying PR3, the two upper tests of Figure 7 satisfy this MR as they show filtering behaviour for frequencies above 0.9 Hz, while the bottom-left plot shows a bandwidth of 0.6 Hz, hence not satisfying this MR. As discussed above, this is due to the appearance of the saturation non-linear

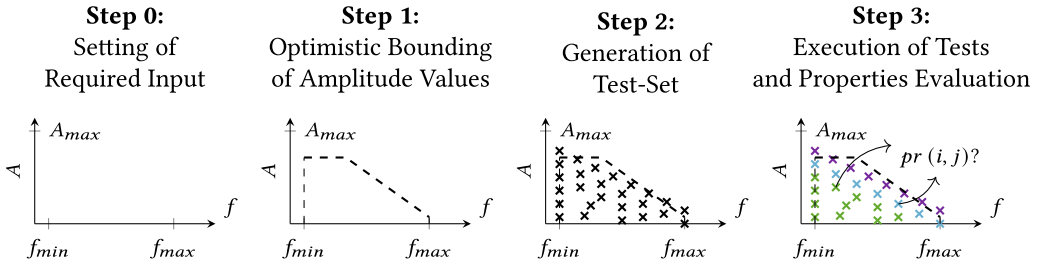


Fig. 9. Graphical representation of main steps of proposed testing approach. For each step, we describe the contribution to the testing in terms of the information that it adds in the frequency-amplitude plane. In each plot, we show the bounds provided by the engineers as required input. The dashed line represents the optimistic bound on the amplitude values obtained in the Step 1. The crosses represent how the tests generated in Step 2 sample the frequency-amplitude plane. In Step 3 the colouring of the crosses and $pr(i, j)$, respectively, exemplify the evaluation of the behaviour observed in the tests and the evaluation of the metamorphic relations.

phenomenon, that however has a limited impact on the CPS and does not push it outside of its design scope.

5.3 Approach Steps

In this section, we describe the main steps of our testing approach. We use our test case parametrisation to generate test cases that cover ranges of amplitudes and frequencies. We then rely on the proposed metrics and MRs enabled by the parametrisation to identify stress test cases.

In our approach, we require the engineers to provide ranges of frequency and amplitude values that are relevant for the SUT. Starting from these ranges, we propose a testing approach based on three main steps. We use Figure 9 to provide an overview and depict how the required input and steps contribute to the generation and identification of stress test cases:

- Step 0. Setting of Required Input.** In this preliminary phase, we ask the engineers to provide bounds on the relevant frequency and amplitude ranges as well as the desired resolution. We use the ranges ($f_{\min}$, $f_{\max}$, and $A_{\max}$ in Figure 9) to limit the scope of the testing to practically relevant values. Furthermore, a resolution δA is also needed to identify the smallest amplitude variations in the reference value that are expected to impact the control behaviour.
- Step 1. Optimistic Bounding of Amplitude Values.** This iterative step provides an optimistic evaluation of amplitude values that cause non-linear behaviour in the SUT (the dashed line in Figure 9). The objective is to obtain a bound that excludes inputs that have such a large amplitude that they are “trivially” outside of the design scope. To obtain such values, we use tests with the sinusoidal shape. The sinusoidal shape maps to a single frequency-amplitude component and does not account for the combination of different components, hence making this step optimistic.
- Step 2. Generation of Test-set.** This step uses different input shapes to generate the actual test set. We use the optimistic amplitude bounds determined by the previous step to limit the target area of the input space during test case generation. For each shape, we use uniform sampling of frequencies and sampling of an exponential distribution over amplitudes to cover the target area. Each of the black crosses in Figure 9 corresponds to a pair of frequency-amplitude coordinates, and hence a generated test.
- Step 3. Execution of Tests and Properties Evaluation.** In this step, we execute the tests and quantify the observed behaviours. The crosses’ colours in Figure 9 depict b_{nl} values, green

being a low value and purple a high value. We use the degree of non-linearity and proposed MRs ($pr(t_i, t_j)$ in Figure 9) to identify test cases that push, to various degrees, the control algorithm out of its design scope.

We now delve into the details of each step.

5.3.1 Step 0: Setting of Required Input. To practically initialise the approach, we ask the engineers to define different quantities specific to the control system. Such quantities initialise the ranges of the amplitudes and frequencies that are relevant for the SUT, and define the desired resolution across tests. The required quantities are:

- A bound on the maximum amplitude value $A_{\max}$ that can be used for the reference¹⁰: this is an upper boundary to practically limit the exploration of high values along the amplitude axis. For example, for drone altitude control, it can be set to the maximum altitude that the drone is expected to fly at.
- A frequency range $[f_{\min}, f_{\max}]$ around the expected closed-loop bandwidth f_b . Such an expected value can be obtained from the control design process or from the speed requirements. As discussed above, it is important to cover the closed-loop bandwidth to push the CPS to its performance limits. As a rule of thumb, a factor of 10 around the expected f_b should be sufficient to make sure to include the actual one. With this rule of thumb, if the tests show that the f_b is not included in the used frequency range, it implies that there is an error of more than one order of magnitude on the expected system bandwidth. Such large error in the expected value suggests the presence of some issue in the system development.
- An amplitude resolution δA that is used to define the smallest variation in the input size that is expected to have an impact on the system behaviour. Ideally, this should be a range within which differences in performance are expected to either not matter or be indistinguishable because of practical limitations (e.g., sensor resolution). However, this is likely to result in a fine-grained resolution that is neither practically needed nor achievable, and larger values can be used.

As we discuss later, we propose a heuristic approach to retrieve a desired frequency resolution from the results of Step 1. In fact, it can be difficult to intuitively define a desired frequency resolution. In case of specific needs, the engineers can set the desired frequency resolution manually (e.g., to control the number of tests given a testing budget). We remark that both the ranges and the resolution can be defined in a conservative way (i.e., using larger ranges and smaller resolution steps) at the cost of increasing the number of tests.

Concerning the approach itself, we require the definition of an upper bound t_{nl} on the degree of non-linearity. This quantity defines a d_{nl} value above which tests are considered to be outside of the design scope of the SUT. Being a threshold on the d_{nl} , it can be interpreted as the maximum relative amplitude of new frequency components that we can accept in the CPS output. Accordingly, it can be chosen based on the maximum relative accepted deviation from the reference value. For example, for drone altitude control, we might be interested in tests that show up to 0.15 m of deviation from the reference when hovering around 1 m, hence setting $t_{nl} = 0.15$.¹¹ Furthermore, its value does not need to be strict. In fact, as far as test case generation is concerned, it is used only for the preliminary optimistic bounding of the amplitude values (Step 1). Hence, a larger value can be selected without compromising the approach effectiveness. However, in the same way as for the ranges and resolutions, a conservative choice for this parameter results in a larger test set.

¹⁰The lower boundary is always zero, since the amplitude describes the absolute value of the signal.

¹¹Note that, d_{nl} being based on the DFT, it has a linear proportion with time-domain values. Hence, relative amplitudes in the two domains are equivalent.

ALGORITHM 1: Step 1: Optimistic Bounding of Amplitude Values

```

1: function OPTIMISTICAMPLITUDEBOUND( $f_{\min}$ ,  $f_{\max}$ ,  $\delta A$ ,  $A_{\max}$ ,  $t_{nl}$ )
2:    $A_{bound} \leftarrow \text{binary\_search\_upperbound}(f_{\min}, \delta A, A_{\max}, t_{nl})$ 
3:    $\text{add\_sample}(\text{upperbound\_amp}, f_{\min}, A_{bound})$ 
4:    $A_{bound} \leftarrow \text{binary\_search\_upperbound}(f_{\max}, \delta A, A_{\max}, t_{nl})$ 
5:    $\text{add\_sample}(\text{upperbound\_amp}, f_{\max}, A_{bound})$ 
6:   while  $\text{max\_gap}(\text{upperbound\_amp}) > \delta A$  do
7:      $f_{next} \leftarrow \text{find\_gap}(\text{upperbound\_amp}, \delta A)$ 
8:      $A_{bound} \leftarrow \text{binary\_search\_upperbound}(f_{next}, A_{\max}, \delta A, t_{nl})$ 
9:      $\text{add\_sample}(\text{upperbound\_amp}, f_{next}, A_{bound})$ 
10:  end while
11:  return  $\text{upperbound\_amp}$ 
12: end function

```

5.3.2 Step 1: Optimistic Bounding of Amplitude Values. The purpose of this step is to obtain, for the different frequency values, an optimistic evaluation of the input amplitude values α that push the control algorithm out of its design scope. With this optimistic evaluation, we can restrict the sampling of the frequency-amplitude plane and avoid trivially large amplitudes that expose non-linear behaviour (i.e., $d_{nl} > t_{nl}$). By excluding such amplitude values, we can focus the test case generation on the area of the input space where the non-linear phenomena start to affect the CPS ability to track reference signals. Intuitively, this restriction of the amplitude values helps avoiding sampling the purple area of Figure 5 and target the azure area.

To obtain this optimistic bound, we test the SUT using sinusoidal inputs ($\phi = \sin$). As described in Section 2.2 (Figure 3), sinusoidal inputs sample a single point in the frequency-amplitude plane and therefore avoid interactions between different components. Since sinusoidal tests do not account for such interactions, they provide only an *optimistic* evaluation of the SUT behaviour for that frequency-amplitude combination. In other words, even if a sinusoidal test with a given value of α and γ exposes linear behaviour ($d_{nl} \leq t_{nl}$), it could happen that other shapes, paired with the same values, expose non-linear behaviour ($d_{nl} > t_{nl}$). On the other end, if a sinusoidal test shows non-linear behaviour, then the use of a different shape (with the same values for α and γ) is unlikely to show linear behaviour. In fact, by changing to a more complex shape, we are adding new frequency components on top of a component that was already sufficient to push the control algorithm out of its design scope.

We show the pseudocode implementing this step in Algorithm 1. The algorithm takes as inputs the frequency range, the desired amplitude resolution, the upper bound on the maximum amplitude value, and the non-linearity upper bound t_{nl} . It then uses tests with sinusoidal inputs to sample the frequency-amplitude plane and obtain a frequency-dependent bound of amplitude values over the frequencies. We represent this bound on the amplitude values with a map object `upperbound_amp` that maps each frequency f to a maximum amplitude value A_{bound} .

In our algorithm, we sample the amplitude and frequency dimensions (γ and α values) with different strategies. We graphically exemplify such strategies in the two plots of Figure 10. For the frequencies, we start by sampling the minimum $f_{\min}$ and maximum frequency values $f_{\max}$ provided in the required input. For each frequency, the algorithm performs a binary search of α values between 0 and $A_{\max}$ (Lines 2 and 4) until it finds the minimum value for which the SUT shows $d_{nl} > t_{nl}$. This binary search is exemplified in the left-hand side plot in Figure 10, where we fix the frequency f and vary the amplitude gain α until we find a test that shows a degree

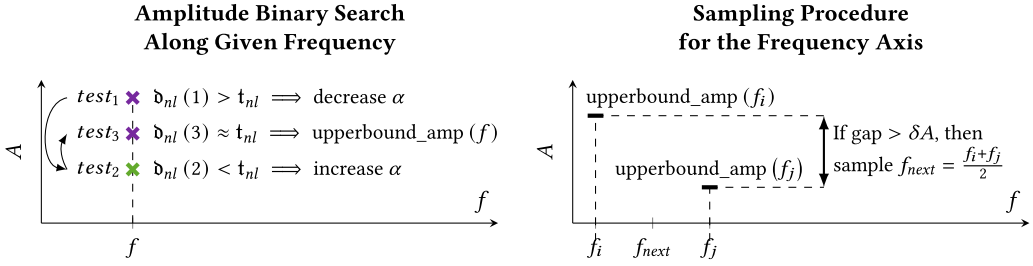


Fig. 10. Graphical examples of the sampling strategies used for the amplitude and frequency axes in Step 1 of the approach (respectively, the `binary_search_upperbound` and `find_gap` functions in Algorithm 1). For exploring the amplitude axis (left-hand side plot), we use a binary search starting from $A_{\max}$ until we find the amplitude for which sinusoidal tests show a d_{nl} close to the chosen threshold t_{nl} . In this example the search takes 3 steps, and performs one test that shows too high d_{nl} ($test_1$), one that shows too low d_{nl} ($test_2$), and a final one close to the threshold ($test_3$). For exploring the frequency axis (right-hand side plot), we exemplify the choice of sampling a new frequency (f_{next}) between two that have already been explored (f_i and f_j). If the upper bound of the two frequencies shows a difference higher than the desired amplitude resolution (i.e., $upperbound_amp(f_i) - upperbound_amp(f_j) > \delta A$), then we chose to sample a new frequency between them (and perform the binary search along the amplitude axis).

of non-linearity close to t_{nl} .¹² In practice, we stop the binary search when the difference between the upper and lower amplitude values identified by the binary search is smaller than the desired amplitude resolution δA (i.e., the next amplitude update is smaller than half the resolution). We then take as A_{bound} the lower amplitude values obtained with the binary search and add it, together with the sampled frequency value, to the map object `upperbound_amp` (Lines 3 and 5).

Then the iterative part of the algorithm starts with the loop at Line 6. In this iteration, we sample the frequency axis until the amplitude value difference between the A_{bound} values of subsequently sampled frequencies are smaller than δA . Such gap is graphically exemplified in the right-hand side plot of Figure 10. In the plot, we have already explored frequencies f_i and f_j , and we evaluate if we need to sample a new frequency between them according to the difference $upperbound_amp(f_i) - upperbound_amp(f_j)$. If there is a gap larger than δA between two subsequent frequencies, then we sample a new frequency f_{next} between them (Line 7). For the new frequency, we perform again a binary search over the amplitude values and add the obtained A_{bound} to the `upperbound_amp` map object (Lines 8 and 9). Once all the amplitude gaps are smaller than δA , we stop the sampling of new frequencies. Finally, the algorithm returns the obtained map object (Line 11). The map object represents the optimistic bound of the amplitude values leading the SUT out of its design scope, and we use it to avoid testing trivially large inputs.

For some systems, it may happen that none of the amplitude values in the range between 0 and $A_{\max}$ causes the SUT to show non-linear behaviour. In such cases, the binary search converges to a value close to $A_{\max}$ and the use of the `upperbound_amp` will not be able to reduce the number of test cases in the next steps of the approach. Such occurrence does not hinder the effectiveness of the approach: it only suggests (but does not imply) that the $A_{\max}$ value might be set too low.

¹²To perform a binary search, we assume that MR1 holds, i.e., that d_{nl} increases for increasing values of α . Thanks to the simple nature of the sinusoidal input, MR1 is more likely to hold for inputs based on this shape. However, as demonstrated in the experimental evaluation (Section 6.4), MR1 does not always hold in practice. The invalidation of MR1 in the preliminary search can lead to overestimate the upper bound; however, such overestimation is still a valid upper bound. In fact, the search stops only around tests with a high d_{nl} value (hence, tests that are in the purple area of Figure 5) or around the maximum allowed value $A_{\max}$. An overestimation implies that the bound is not as tight, thus rendering the approach less efficient. Specifically, it increases the number of tests needed to cover the area below the bound.

5.3.3 Step 2: Generation of Test-set. In this step, we generate the actual test set. Given our test case parametrisation, we need to define a set of shape functions and different amplitude and time-scaling parameter values. We now discuss each of them.

Concerning the shape functions, we propose to use a set of shapes inspired by common practice in control engineering. In control engineering, the most common inputs used to evaluate a control algorithm are the step (instantaneous change of the reference) and ramp (linear change in the reference) [3]. Accordingly, we propose the use of shape functions that resemble such inputs: square, sawtooth, triangular, and trapezoidal waves. If other patterns are available from use cases in a specific CPS application, then engineers can expand this set.

For each of the chosen shapes, we generate a set of $\langle \gamma, \alpha \rangle$ pairs. We aim at exploring the area of the input space delimited by the upper-bound threshold identified in Step 1. Since one shape corresponds to more than one frequency-amplitude $\langle f, A \rangle$ point, we take as reference the point associated with the largest amplitude, which we call the *main component* of a shape. Accordingly, we sample values for α and γ and thus move the main component of each shape to obtain test cases that cover the area delimited by the upper-bound threshold.

Regarding the frequency axis, in the general case, we assume that there is no reason to test a specific range. Therefore, we aim at uniformly covering (with the main component of the shape) the frequency range $[f_{\min}, f_{\max}]$. Practically, we sample frequencies at equal intervals in the given range. Since it can be difficult to intuitively set the frequency resolution, we propose to compute such resolution δf from the results of Step 1. Specifically, we suggest to use the average frequency gap (i.e., $\text{avg}_i \{f_{i+1} - f_i\}$) obtained in the amplitude upper-bound. Intuitively, these gaps are obtained by imposing a maximum difference δA in the amplitude threshold for which the SUT shows non-linear behaviour. Therefore, they should correspond to frequency variations for which the $\mathfrak{d}_{nl}$ does not change significantly. In specific cases, the engineers can adapt the sampling of the frequency axis according to application-specific needs. For example, if a frequency range is known to be particularly relevant for the specific SUT, a biased random sampling can be applied.

Differently, along the amplitude dimension, we are generally more interested in exploring the area with large amplitudes. For the amplitude values, we can leverage the upper bound obtained at Step 1 and avoid sampling large amplitudes that, already with a sinusoidal input, would provide a very high $\mathfrak{d}_{nl}$. Practically, we limit the sampling range with the optimistic upper bound obtained in each sampled frequency. Within this range, we propose to sample different amplitudes α according to a beta distribution skewed toward the higher values [50].¹³ The number of samples is determined according to the desired amplitude resolution δA . Practically, we compute the number of tests for each frequency f with $\text{upperbound_amp}(f) / \delta A$ to obtain a number of samples compatible with the desired resolution. Similar to the frequency dimension, application-specific sampling strategies might be adopted if areas of particular interest are given.

5.3.4 Step 3: Execution of Tests and Properties Evaluation. In this final step, we proceed to executing the tests and evaluating their outputs. The evaluation is based on the $\mathfrak{d}_{nl}$ and $\mathfrak{d}_f$ metrics defined in Section 4.2 and includes the verification of the MRs and the identification of the stress test cases. We first discuss the identification of stress test cases and evaluation of MR1 using $\mathfrak{d}_{nl}$. We then discuss the evaluation of MR2 and MR3 using $\mathfrak{d}_f$. We use Figure 11 to exemplify the manual evaluation of the MRs. In the experimental section, we showcase in detail the visualisation and analysis step for our case studies.

We execute each test and store its output trace. We then compute, according to the definitions given in Section 4.2: (i) the set of relevant $\langle f, A \rangle$ components, (ii) the degree of non-linearity,

¹³A beta distribution is a version of the more classical exponential distribution with bounded support.

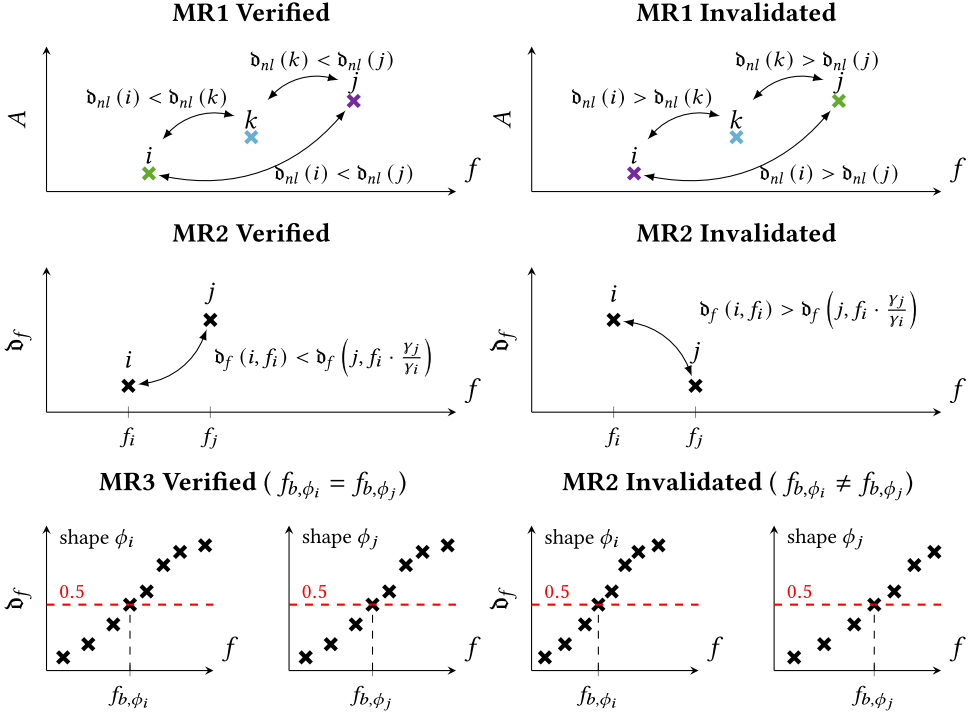


Fig. 11. This figure shows how to manually evaluate the proposed MRs. For each MR (the different rows), we report an example in which it is verified (left column) and one in which it is invalidated (right column). To evaluate **MR1**, we compare the d_{nl} measured in different tests (tests i , j , and k) and their relative position in the frequency-amplitude plane. We compare the d_{nl} for each pair of tests (highlighted by the arrows in the plots). If the d_{nl} is higher for tests further away from the origin (left-hand side plot), then MR1 is verified. If the d_{nl} decreases for tests further away from the origin (right-hand side plot), then MR1 is invalidated. To evaluate **MR2**, we compare the d_f in frequency components of tests (tests i and j) with different time-scaling coefficients ($\gamma_i > \gamma_j$). If a larger time-scaling coefficient corresponds to a higher d_f (left-hand side plot), then MR2 is fulfilled. If a larger time-scaling coefficient corresponds to a smaller d_f (right-hand side plot), then MR2 is invalidated. To evaluate **MR3**, we compare the closed-loop bandwidth (the frequency for which the d_f becomes greater than 0.5) using tests with different shapes (shapes ϕ_i , and ϕ_j). If the closed-loop bandwidth evaluated with tests that use different shapes are equal (left-hand side plot $f_{b,\phi_i} = f_{b,\phi_j}$), then MR3 is verified. If the two closed-loop bandwidths evaluated do not correspond (right-hand side plot $f_{b,\phi_i} \neq f_{b,\phi_j}$), then MR3 is invalidated.

and (iii) the degree of filtering of each $\langle f, A \rangle$ component. Such information can then be analysed by leveraging plots on the frequency and amplitude dimensions.

First, we discuss how to use d_{nl} to identify stress test cases and evaluate MR1. To do so, the main component of each test (i.e., the component with the largest amplitude) can be displayed in a scatterplot on the frequency-amplitude plane using its $\langle f, A \rangle$ coordinates. The markers can then be coloured with a gradient that represents the measured d_{nl} (e.g., purple for $d_{nl} \geq t_{nl}$, and green for $d_{nl} = 0$, with the colour gradients for representing the values in the range $[0, t_{nl})$). We exemplify such scatterplot with the mentioned colour choice in the upper plots of Figure 11. The engineers can then use the colours to distinguish the input space regions within the design scope of the CPS (green regions characterised by a low d_{nl}) and regions outside of the CPS design scope (purple regions characterised by a high d_{nl}). The test cases at the borders between the regions where we

observe a transition between colours are the stress test cases. The inspection of the stress test cases provides information about the phenomena in the CPS that can falsify the design assumptions and push the system out of its design scope.

- **MR1** can be evaluated analysing the relative positioning of regions characterised by low and high $\mathfrak{d}_{nl}$. In fact, MR1 states that $\mathfrak{d}_{nl}$ should increase with increasing frequency and amplitude values. Practically, regions with low $\mathfrak{d}_{nl}$ (green regions) should be closer to the origin of the axes than regions with high $\mathfrak{d}_{nl}$ (purple regions). Accordingly, a pattern similar to the one shown in Figure 5 should appear. For example, in the upper left-hand side plot of Figure 11 where the tests i , k , and j show increasing $\mathfrak{d}_{nl}$, consistent with their distance from the origin (respectively, green, azure, and purple colouring).

If the test results show such patterns, then they fulfill MR1.

We now discuss the evaluation of MR2 and MR3. As discussed in Section 5.2, MR2 and MR3 describe expected $\mathfrak{d}_f$ properties for tests within the design scope (i.e., tests for which $\mathfrak{d}_{nl} < t_{nl}$). Accordingly, we use these MRs to validate the identification of tests that are within the design scope: if such identification is correct, then the tests should fulfil the two MRs, otherwise there is likely some issue with the testing process.

To discuss their evaluation, we note that MR2 and MR3 (i) concern only tests that show linear behaviour, (ii) do not depend on the input amplitude, and (iii) address $\mathfrak{d}_f$.

Accordingly, to evaluate them, we first select only the tests that fulfil $\mathfrak{d}_{nl} < t_{nl}$, i.e., the tests that are considered within the design scope. From such tests, we extract the individual $\langle f, A \rangle$ components and associated $\mathfrak{d}_f$ value. We then plot, in separate plots for each shape, $\mathfrak{d}_f$ of each frequency-amplitude point as a function of its frequency coordinate f . We exemplify such plotting in the middle and bottom plots of Figure 11, where the crosses represent a frequency-amplitude component with the horizontal coordinate being the corresponding frequency value (f_i and f_j) and the vertical coordinate representing $\mathfrak{d}_f$ measured for that component. Such plots can then be used to evaluate MR2 and MR3:

- **MR2** states that an increase in the frequency values corresponds to an increase in the degree of filtering. Accordingly, the proposed frequency- $\mathfrak{d}_f$ plot should show an increasing trend irrespective of the used shape. For example, in the middle plots of Figure 11 the left-hand side one shows an increasing pattern that fulfils MR2, while the right-hand side one shows a decreasing pattern that invalidates MR2.
- **MR3** states that f_b should not depend on the specific shape of the input. We recall that f_b is defined as the frequency below which $\mathfrak{d}_f < 0.5$ and above which $\mathfrak{d}_f > 0.5$. For example, in the bottom plots of Figure 11, we use shapes ϕ_i and ϕ_j and observe the closed-loop bandwidths f_{b,ϕ_i} and f_{b,ϕ_j} . Accordingly, the trend of the degree of filtering values for the different shapes should cross the 0.5 value around the same frequency. In Figure 11, the left-hand side plots show $f_{b,\phi_i} = f_{b,\phi_j}$ and fulfils MR3, while the right-hand side plots show $f_{b,\phi_i} \neq f_{b,\phi_j}$ and thus invalidates MR3.

If the above properties are not verified by some frequency-amplitude components, then the tests associated to such components should be inspected for identifying possible faults in the testing process.

6 EMPIRICAL EVALUATION

In this section, we empirically evaluate the proposed testing approach. We aim to assess its effectiveness in using the frequency and amplitude dimensions for generating test cases that falsify the linearity assumptions made during the control algorithm design. Furthermore, we

want to evaluate the ability of the MRs to highlight test cases that fall outside of the design scope and provide insights about the specific non-linear phenomena that can lead to the SUT behaving unexpectedly. Accordingly, we define two **research questions (RQ)**.

RQ1—Generation of Stress Tests: We aim to evaluate how effective is the use of the frequency and amplitude characterisation of the test inputs to trigger non-linear phenomena that limit the design scope. With RQ1, we ask: *How effective is our test case generation approach in pushing a CPS control-loop away from the linearisation point and triggering non-linear phenomena around the boundary where they appear?*

RQ2—Relevance of Metamorphic Relations: With the second research question, we aim to evaluate the possibility of using the proposed MRs to identify inconsistencies with the qualitative input space characterisation and thus identify which tests stress the system such as to induce non-linear behaviour. Hence, we ask: *How the proposed MRs identify stress test cases at the bounds of the SUT design scope?*

This section is structured as follows. We first present and motivate the case studies (Section 6.1) as well as the settings choices required for our testing approach (Section 6.2). Then, for each question, we present our empirical methodology, followed by the evaluation results, that we use to answer the RQs (Sections 6.3 and 6.4). We conclude this empirical evaluation with a discussion of the threats to validity (Section 6.5), and information about code and data availability (Section 6.6).

6.1 Systems Under Test

To answer our research questions, we apply our testing approach to the simulation models of three CPSs: a Python model of the Crazyflie drone, a Simulink model of a DC servo (a continuous current motor), and a Simulink model of a lightweight aircraft. We choose these CPSs based on three main criteria. First, we aim at covering the two most common types of algorithms found in traditional control theory: PID control and state feedback [3, 18]. The drone and the lightweight aircraft include PID control algorithms, while the DC servo uses state feedback. Intuitively, the difference between the two is that PIDs compute the actuation based on the distance between the physical output and the desired value, while state feedback bases the computation on the whole system state estimation.

Second, we need SUTs that include non-linear phenomena as well as detailed knowledge about these phenomena (e.g., saturation bounds) and access to their development documentation. This knowledge is necessary to evaluate the effectiveness of our approach in triggering such phenomena. Accordingly, for the Crazyflie drone and the DC servo, we use simulation models that we built based on their development documentation and actual CPS implementations, which are available to us. In this way, we can also build models behaving consistently with the real systems. However, to gain further insights and higher confidence in the general validity of our approach, we also include a CPS model that was not built by us. We chose the publicly available model of the altitude control of a lightweight aircraft.¹⁴ This model fits our needs as it includes both a detailed model of the aircraft dynamics (hence including the non-linear phenomena) as well as the control algorithm implementation.¹⁵

Third, we want our case studies to capture the variability that can appear among SUTs when applying our testing approach. In fact, when applying our testing approach, engineers cannot

¹⁴<https://nl.mathworks.com/help/aeroblks/lightweight-airplane-design.html>

¹⁵As discussed at the end of Section 3.1, in the scope of this article, we need more detailed models than the ones used by control engineers. This limits our choice of publicly available models as many of the publicly available ones do not have such level of detail (e.g., <https://nl.mathworks.com/help/simulink/slref/aircraft-longitudinal-flight-control.html>)

know *a priori* which non-linear phenomena affect the SUT design scope. For this reason, we want to include different types of non-linear phenomena in our case studies. In this regard, the DC servo, being an electric motor, has a wide variety of applications, and depending on the specific application it can expose different non-linear phenomena. This feature of the DC servo allows us to inject different non-linearities that are plausible in a specific application context. In this way, we can obtain the desired SUT variability.¹⁶ We now briefly describe the three CPSs as well as their simulation models, including information on how we built the Crazyflie drone model and the different versions of the DC servo.

The Crazyflie drone is developed by Bitcraze¹⁷ and is completely open-source. This gives us full access to the source code, as well as software and design documentation [23]. We use the documentation to build a detailed Python model of the physics. In addition, we use the source code together with the design documentation to build the model of the control software.¹⁸ In previous research, we compared the flight of the model with the flight of the real drone and observed that they show similar behaviour [34]. This gives us confidence on the real-world validity of the experiments performed with this model. The control layer of the drone includes three separate control loops for the three directions. Among those, we chose to test the altitude control, which controls the drone's vertical position as, different from the other control loops, it allows for the quantification of the non-linear phenomena occurrence that we leverage in answering RQ1. Despite being a small drone (size of 6×6 cm), we note that the Crazyflie drone implements standard drone control algorithms that are also used in larger drone applications [42].

The Simulink model of the DC servo is based on laboratory hardware used for educational purposes at Lund University. Hence, like for the Crazyflie drone, we have access to the CPS implementation and its documentation. The control layer of the DC servo receives from the users a desired angular position of the motor axis (the input reference r). Given such input references and the motor axis position measurements (the output y), the control layer generates a voltage signal to be fed to the motor to make it move (the actuation signal). We built the Simulink model to execute the very same C code that is used in the microcontroller of the DC servo. This gives us confidence that the simulation model behaves similarly to the actual implementation. While the considered DC servo is used for educational purposes, it is analogous to electrical motors found in many industrial applications such as the joint of a robot or the steering axis of an autonomous car. As mentioned above, we chose this SUT because electrical motors can expose, depending on the specific application, different types of non-linearities that can affect the behaviour of the control algorithm. This allows us to develop different but still practically relevant versions of the DC servo in which we manually inject non-linearities to study their impact. To ensure these non-linearities are commonly found in real applications, we use the following non-linearities defined in the "Discontinuities" folder of the Simulink library¹⁹:

- *Saturation* represents the limitation of variables in the system. It generally affects actuators (which have limited capacity) and sensors (which typically have a limited range of values they can read).
- *Quantisation* is the effect of the finite precision in the analog-to-digital conversion of the sensors signals.
- *Pulse-Width-Modulation* is a common technique used for digital-to-analog conversion, which can distort the actuation signals.

¹⁶Conversely, we decided not to manually inject non-linearities in the model of the Crazyflie drone to keep the SUT identical to the one used during the actual development of the drone.

¹⁷<https://www.bitcraze.io/>

¹⁸<https://github.com/bitcraze/crazyflie-firmware>

¹⁹<https://www.mathworks.com/help/simulink/discontinuities.html>

- *Coulomb Friction* is always found in the physical world but is however hard to model and non-linear and, for these reasons, often neglected or approximated.
- *Dead-Zone* and *Backlash* are always found in mechanical gearing as they are caused by the play between the different cogs.²⁰

We add to this list a quadratic friction model as, in practice, it is a very common source of non-linear behaviour, e.g., aerodynamic drag in a car or drone. Among these non-linearity sources, we consider saturation, quantisation, and pulse-width-modulation to be always present as they are part of the interface between the cyber and physical parts of the system. In contrast to these sources, we inject one by one the quadratic friction, the coulomb friction, the dead-zone and backlash. We therefore obtain the following five versions of the DC servo according to the included non-linearities:

- DC1** saturation (on actuation and sensor), quantisation, pulse-width-modulation.
- DC2** saturation (on actuation and sensor), quantisation, pulse-width-modulation, and Coulomb friction.
- DC3** saturation (on actuation and sensor), quantisation, pulse-width-modulation, and quadratic friction.
- DC4** saturation (on actuation and sensor), quantisation, pulse-width-modulation, and dead-zone.
- DC5** saturation (on actuation and sensor), quantisation, pulse-width-modulation, and backlash.

Finally, in the altitude control of the lightweight aircraft, the objective is to reach and maintain a desired altitude. To achieve this, the software is responsible for setting the elevator angle (one of the moving surfaces of the aircraft tail) based on various sensor data such as air pressure and gyroscope readings. This CPS, as noted above, was not developed by us, and we do not have detailed knowledge on the non-linear phenomena that it entails. This limits our ability to quantify the occurrence of non-linear phenomena (RQ1) and of identifying root causes for the misbehaviour observed in the tests. However, it does not impact our findings in terms of identification of the test cases where non-linear phenomena impact the CPS ability to track references (RQ2). To summarise, we apply our approach to seven different SUTs: the two altitude control models for the drone and the aircraft, and the five different versions of the DC servo.

6.2 Testing Settings

In this subsection, we present the settings needed for the application of our testing approach to the selected SUTs. We first report on the required input (defined in Section 5.3) and the number of input repetitions needed to evaluate the d_{nl} metric (defined in Section 4.2). We conclude by reporting the resulting number of executed tests.

Selection of Required Inputs: We summarise in Table 1 the values chosen, based on our domain knowledge, for the required inputs of our testing approach.

Concerning the frequency range, the drone and the DC servo should be able to track the reference within a time span in the order of seconds. Accordingly, we place the expected closed-loop bandwidth for these SUTs around 1 Hz. Intuitively, this corresponds to taking around 2 s to track a unit change in the reference value (e.g., 1 m for the drone and 1 rad for the DC servo). For the drone, we define the frequency range to be [0.1 Hz, 2 Hz]. For the DC servo, we do not have a clear estimate from its development documentation. Hence, we choose a broader range [0.005 Hz, 3 Hz].

²⁰ Among the blocks listed in the Discontinuities folder, we excluded the “rate limiter” as it is just a special case of saturation, and the “relay” as it is a very simple and old control approach, used only for non-critical systems.

Table 1. Selected Input Values for Our Testing Approach to Test the Drone, the DC Servo, and the Lightweight Aircraft

SUT	$f_{\min}$	$f_{\max}$	$A_{\max}$	δA
Crazyflie Drone	0.1 Hz	2 Hz	6 m	0.05 m
DC servo (all versions)	0.005 Hz	3 Hz	2π rad	0.015 rad
Lightweight Aircraft	0.005 Hz	0.5 Hz	750 m	5 m

The lightweight aircraft is a slower system, and we expect the bandwidth to be around 0.05 Hz, which gives a response to a unit change in the reference of the order of tens of seconds. Accordingly, using the rule of thumb of including one order of magnitude above and below the expected closed-loop bandwidth, we choose the frequency range [0.005 Hz, 0.5 Hz].

Concerning the maximum amplitude, we reason on the largest inputs that we can expect in practice for our SUTs. Since the Crazyflie drone is expected to mostly fly in indoor environments, we chose a maximum amplitude of 6 m, corresponding to a large room.²¹ The DC servo is supposed to track desired angular positions. Hence, we limited the reference amplitude changes to one full rotation of the motor axis, 2π rad. The lightweight aircraft is expected to fly around 2,000 m in altitude and perform manoeuvres around that height. Hence, we set a maximum amplitude range for the manoeuvres of 750 m, resulting in references between 1,250 and 2,750 m.

Concerning the amplitude resolution, we reason about the variations of the input amplitude that can cause a change in the behaviour of the SUT. For the drone, this is in the order of centimetres. For example, asking the drone to reach 1.01 or 1 m should not show significantly different behaviour. Accordingly, we chose $\delta A = 5$ cm for the drone. The DC servo is expected to react to reference changes in the order of rotation degrees. Since such devices are expected to be rather precise, we chose a range smaller than the desired precision. Specifically, we chose $\frac{1}{400}$ of a rotation for the DC servo (less than one degree): $\delta A = 0.015$ rad. For the lightweight aircraft, we can expect that a difference in the reference in the order of meters does not significantly alter the system behaviour; hence, we chose $\delta A = 5$ m.

Finally, to apply the testing approach to the SUTs, we have to set the non-linear threshold. The t_{nl} threshold is connected to the maximum relative deviation from the reference that the SUTs can accept. In our experiments, across the SUTs, we observed unexpected behaviours when output signals deviate 15% from the reference r . Accordingly, we set $t_{nl} = 0.15$. This means that we identify tests with more than 15% deviation from the reference r to be trivially out of the design scope and therefore not interesting in terms of stress testing. For example, in the drone case study, this means that we are interested in tests that deviate of up to 15 cm when the drone is expected to hover at 1 m of height.

Setting the Number of Input Iterations: To apply our testing approach to the SUTs, we need to define the number of input repetitions needed to compute the $\mathfrak{d}_{nl}$ metric (Section 4.2). For efficiency, we want to select the smallest number of repetitions (i.e., shorter tests) yielding sufficient accuracy when computing $\mathfrak{d}_{nl}$. To evaluate this trade-off, we performed a preliminary experiment, in which we ran a single long test with the maximum amplitude and time-scaling coefficients (corresponding to $A_{\max}$ and $f_{\max}$ in Table 1) such that the SUT exposes non-linear behaviour. We detect non-linear behaviour by inspecting whether the output follows the input reference. We use 10 input repetitions and compute the $\mathfrak{d}_{nl}$ value using different trace lengths that correspond to different numbers of input periods. We then evaluate after how many input periods the $\mathfrak{d}_{nl}$ value converges to a value detecting non-linear behaviour (i.e., it is higher than the chosen t_{nl}). As this

²¹For example, Crazyflie has been used for automated inventorying of supermarkets [24].

Table 2. $\mathfrak{d}_{nl}$ of the Preliminary Tests Computed Using Different Numbers of Periods for Each SUT

Number of Periods	1	2	3	4	5	6	7	8	9	10
Crazyflie Drone	0.12	0.26	0.36	0.40	0.39	0.41	0.47	0.54	0.56	0.54
DC1	0.05	0.10	0.09	0.09	0.08	0.50	0.45	0.42	0.37	0.30
DC2 (Coulomb Friction)	0.05	0.10	0.09	0.08	0.07	0.54	0.49	0.47	0.43	0.36
DC3 (Quadratic Friction)	0.01	0.02	0.03	0.01	0.02	0.25	0.24	0.24	0.25	0.24
DC4 (Dead-Zone)	0.05	0.10	0.09	0.09	0.07	0.49	0.45	0.44	0.41	0.35
DC5 (Backlash)	0.05	0.11	0.10	0.09	0.07	0.49	0.45	0.42	0.38	0.32
Lightweight Aircraft	0.00	0.01	0.02	0.04	0.05	0.08	0.10	0.14	0.17	0.22

For the drone, we can see that the value stabilises when using five periods; hence, that is the number of periods that we use when applying our approach to this SUT. For each of the DC servo versions, the value detects the non-linear behaviour after six repetitions; hence, we use seven to allow for some margin. For the lightweight aircraft, we see that the $\mathfrak{d}_{nl}$ value steadily increases up to ten input repetitions and is greater than the chosen threshold for nine or more repetitions; hence, we chose ten input repetitions.

evaluation is different for the different SUTs, we run it for each of them before applying our testing approach.

To manually bring the SUTs out of their design scope, we use tests with high frequency and amplitude. As discussed in the $\mathfrak{d}_{nl}$ definition (Section 4.2.2), the specific amplitude, frequency, and shape used for this preliminary experiment are not relevant as long as the test pushes the SUT out of its design scope. We run tests with the specified maximum amplitude and frequencies (Table 1). To cause non-linear behaviour, for the drone and the lightweight aircraft, we used a test with the sinusoidal shape, while for the DC servo, we used a test with the steps shape.

We report in Table 2 the $\mathfrak{d}_{nl}$ values obtained for each of the SUTs using different numbers of input periods. For the drone, the $\mathfrak{d}_{nl}$ value exceeds the threshold already when using two periods. We chose, however, to use five periods as $\mathfrak{d}_{nl}$ increases until then. For the DC servo, we observe a similar pattern in each version of the system. The $\mathfrak{d}_{nl}$ value is below the threshold when using 5 or less periods and then suddenly increases to a higher (and constant) value above the threshold for 6 or more periods. To allow some margin, we chose to use 7 periods in our tests. For the lightweight aircraft, the $\mathfrak{d}_{nl}$ value passes the threshold for 9 periods; hence, for the same reason as for the DC servo, we use tests with 10 periods.

We note that the measured $\mathfrak{d}_{nl}$ does not always increase when we increase the number of periods used for its computation (e.g., for 6 to 10 periods in the **DC4**). In general, the more frequency samples we have, the more likely we are to detect new frequency components in the output signal and hence having a higher $\mathfrak{d}_{nl}$. However, when increasing the length of the traces, the new frequency samples move on the frequency axis, as shown in Figure 8. Since different frequencies are sampled, different amplitudes can be measured, and hence, there is no guarantee that the $\mathfrak{d}_{nl}$ value increases monotonically.

Number of Tests and Duration of Experiments: In Table 3, we report the number of tests executed for each SUT. The table reports the number of tests executed, respectively, for Step 1 and Step 3.²² We note that the DC servo requires more tests as it has wider ranges and a higher resolution was selected. For the drone and lightweight aircraft, the execution of Step 1 reduces the number of tests needed to cover their input space. In total, for our experimental campaign, we executed 30,187 tests.²³

The execution of the tests took few hours for the drone and the lightweight aircraft on a 2017 MacBook Pro, with 2.3 GHz Dual-Core Intel Core i5 processor. The different versions of the DC

²²Recall from Section 5.3 that Step 2 does not require the execution of test cases.

²³In Section 7, we comment on what insights the number of executed tests provides on the testing approach.

Table 3. Number of Tests Executed for Each SUT in Step 1 (Optimistic Bounding of Amplitude Values) and Step 3 (Execution of Tests and Properties Evaluation)

Approach Step	Step 1	Step 3	Total
Crazyflie Drone	112	1, 012	1, 124
DC1	12	5, 100	5, 112
DC2 (Coulomb Friction)	12	5, 010	5, 022
DC3 (Quadratic Friction)	12	5, 100	5, 112
DC4 (Dead-Zone)	12	5, 100	5, 112
DC5 (Backlash)	12	5, 100	5, 112
Lightweight Aircraft	172	3, 421	3, 593

servo took instead almost a full day for each version. This is due both to the higher number of tests and also to the pulse-width modulation. In fact, the simulation of the pulse-width modulation includes a high-frequency signal that requires a very short simulation time step, hence making the simulation significantly slower.

6.3 RQ1—Generation of Stress Tests

Methodology: To answer this question, we define metrics to evaluate the occurrence of the different non-linear phenomena in the performed tests. We study tests results over the different values of the proposed frequency-amplitude input characterisation. If our approach is effective, then the tests results will show various occurrences of the non-linear phenomena depending on the frequency and amplitude content of the input. Such tests can then be used to assess how and in which scenarios the non-linear phenomena affect system performance. For the Crazyflie drone and the DC servo, we have complete knowledge of the non-linear phenomena involved. Hence, we can conduct an exhaustive evaluation of their results with respect to this research question. For the lightweight aircraft, however, we do not have such detailed knowledge about its non-linear phenomena. Upon inspecting the Simulink model of the aircraft, we could identify non-linearity due to actuator saturation. Therefore, we limit ourselves to evaluating the occurrence of this non-linear phenomenon. We now discuss how we use the knowledge about the SUTs to define metrics that quantify the occurrence of non-linear phenomena.

In the case of the Crazyflie drone, we identify two non-linear phenomena: (1) the saturation of the actuators, which occurs as the electric motors have a constrained power range and (2) the fact that the motors cannot generate negative force, meaning that the propellers cannot generate a force that pulls the drone down. Hence, downward movement is achieved only through gravity force. Both phenomena can be detected when the voltage signal sent to the motors reaches its upper or lower limit. Accordingly, to quantify the occurrence of non-linear phenomena for a given test, we calculate the percentage of test time during which the actuators are saturated (i.e., when they have reached their upper or lower limit).

As for the DC servos, non-linear phenomena include saturation (of both actuators and sensors), pulse-width modulation, quantisation, and injected non-linearities. Analogously to the drone, we quantify saturation with the percentage of test time during which they occur. However, pulse-width modulation and quantisation affect every sensor reading and actuation in the control loop. Hence, it is not possible to distinguish tests in which they appear more than others. Regarding the non-linearities injected in separate instances of the SUT, we quantify them as the deviation from how the model would have behaved in their absence. For example, when we inject non-linear models of friction, we can compare them to the friction value that we would have obtained with

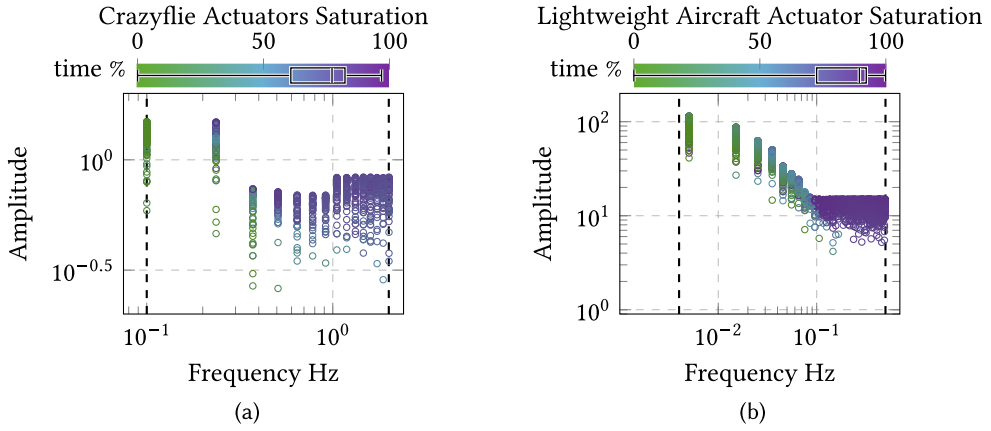


Fig. 12. These figures present the quantified occurrences of the actuator saturation, respectively, in the drone tests (plot on the left) and in the lightweight aircraft tests (plot on the right), measured using the defined metrics. On the frequency-amplitude plane, each test is captured by a single point corresponding to the main component of the input sequence. For each point (i.e., each circular marker), the colour gradient indicates the quantified occurrence of the actuator saturation, where green corresponds to no actuator saturation (0% of test time) and purple corresponds to complete saturation (100% of test time). The boxplot overlaid on the colourbar highlights the median, minimum, maximum, and quartiles of the occurrences of non-linear phenomena in the tests.

a linear model. Accordingly, at every time instant, we measure the difference between the two values. We then average such values over the whole test and obtain a metric capturing the degree of observed non-linearity.²⁴

Finally, as mentioned above, we do not have the same level of detailed knowledge about the non-linear phenomena involved in the lightweight aircraft as we do for the Crazyflie drone and the DC servo. However, we could identify non-linearity caused by actuator saturation based on our inspection of the model. This saturation limits the angle that the elevator (the moving surface of the aircraft tail) can take. In the same way as for the other case studies, we quantify the occurrence of actuator saturation in a test as the percentage of time that the elevator is at the maximum or minimum angle.

Reading Guide for Non-linear Phenomena Occurrence Plots: We report the measured occurrence of the different non-linear phenomena in Figure 12(a) for the drone, Figure 12(b) for the aircraft, and Figure 13 for the different versions of the DC servo. For the drone and the aircraft, we report plots on the occurrence of actuator saturation. For the DC servo, in Figure 13, the rows correspond to the different versions of the SUT. The columns correspond to the three non-linear phenomena: the actuator and sensor saturations, and the injected non-linearity. Each plot shows the quantification of a non-linear phenomenon occurrence in the frequency-amplitude plane. For each test, we use the frequency-amplitude values of the test's main component as coordinates to position the test in the plane. We then use the colour gradient of the marker (empty circle)

²⁴We note that this approach can give very different value ranges depending on the specific phenomenon considered. For example, the input variation caused by the play found between cogs, i.e., the backlash or the dead-zone, will be much smaller in value than the friction variation caused by the use of a quadratic model instead of a linear one. However, regarding the answer to RQ1, we are only interested in the definition of a value that is zero when the model is behaving linearly and that increases when the behaviour differs further from the linear model. We are not interested in a comparison of the absolute values. Hence, it is not necessary to normalise the data.

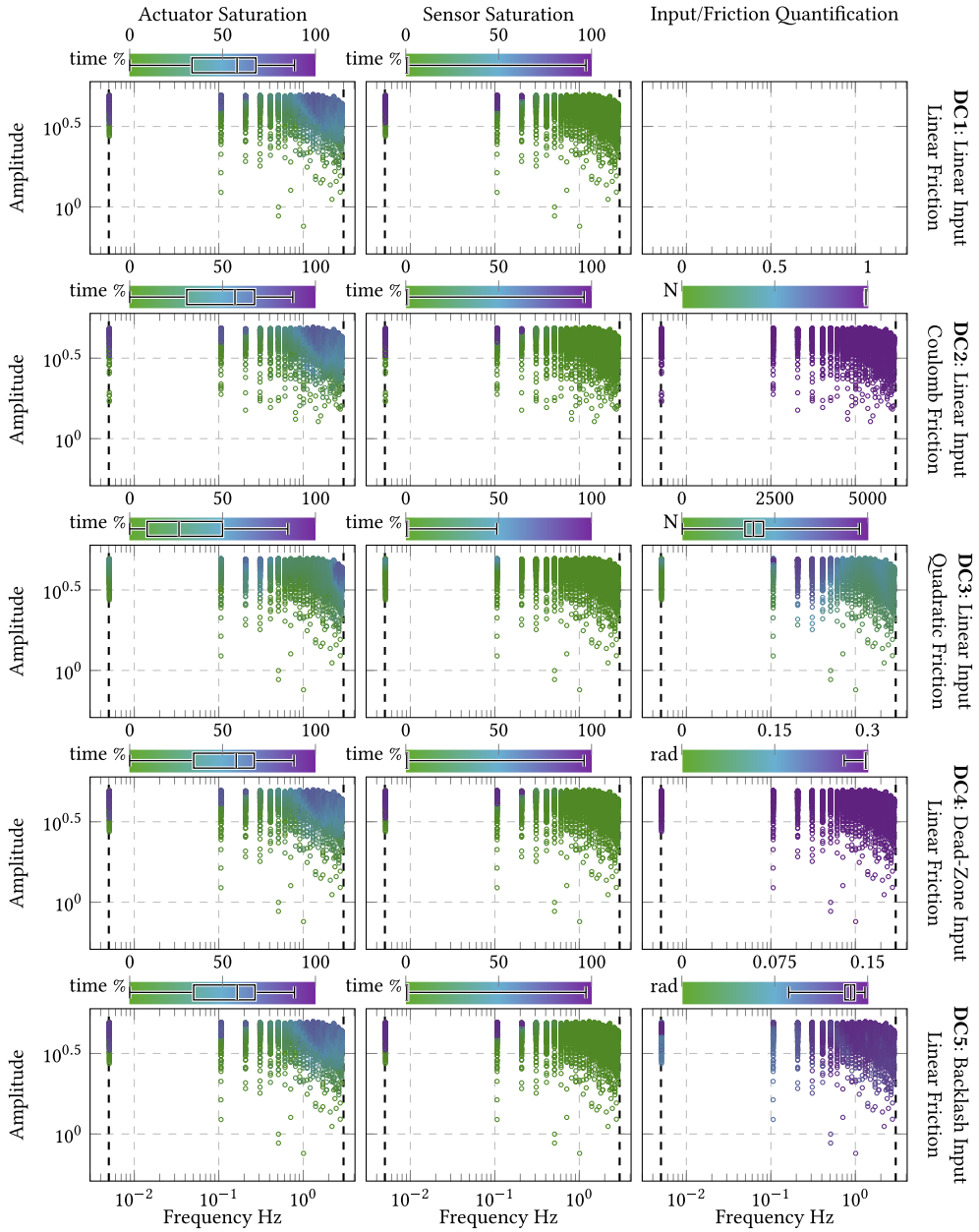


Fig. 13. This figure presents the quantified non-linear phenomena occurrences in the different DC servo tests, measured using the defined metrics. Each row corresponds to a version of the DC servo. The columns correspond to the different non-linearities: the first two show the actuator and sensor saturation analogously to Figure 12. The rightmost column shows the injected non-linearity in the friction (second and third rows) or in the actuation (fourth and fifth rows). The first row reports the DC servo without injected non-linearity, hence the plot is empty. For each marker, the colour gradient indicates the quantified non-linear phenomenon occurrence: green are zero values and purple are higher ones. The boxplot overlaying the colourbar highlights median, minimum, maximum, and quartiles of the non-linear phenomena occurrence in the tests.

for representing the quantified occurrence of the non-linearity phenomenon. Green corresponds to a zero value of the quantified occurrence, and purple corresponds to the maximum measured value. Above each plot, we report the colour-scale mapping the occurrence values with the colours. Furthermore, we use the boxplot convention to represent the median, minimum, maximum, and quartiles of the occurrences of each non-linear phenomenon, overlaid on each of the colourbars reporting the gradient scale. While we are mostly interested in the relationship between the occurrence of non-linear phenomena and the frequency-amplitude content of the input, the boxplots provide better visibility into the frequency of a given non-linear phenomenon in our tests. In other words, they show how many green or purple points we have in each plot, corresponding to the occurrence of non-linear phenomena.

Results: Using such figures, we now discuss the ability of the test approach to expose different non-linear phenomena. The *actuator saturation* non-linearity concerns all of our SUTs. From Figures 12(a), 12(b), and the left-most column of Figure 13, we observe that our tests cover a wide range of saturation percentages for each of our SUTs. This can be seen from the fact that the scatter plots cover the full scale of the colour gradient and confirmed with the boxplots above them. From the boxplots for the DC servo tests, we can observe that our approach only misses tests exposing more than 90% of saturation time. This is due to the range of input values considered (one rotation of the motor) combined with the use of a powerful motor. Such tests could be possibly covered with a higher value for $A_{\max}$: intuitively, higher reference values require higher actuation. However, such tests would not be interesting as they would go beyond the range of possible angles. This shows that our testing approach is able to generate test cases that trigger the saturation of actuators to diverse degrees. Furthermore, from the frequency-amplitude scatter plots, we can highlight that, by covering different amplitude and frequency values, the approach is covering the transition areas of the frequency-amplitude plot where actuator saturation starts to appear. As predicted, higher values of frequency and amplitude content, as well as the combination of the two, tend to cause the actuator saturation to appear.

When looking at the *sensor saturation*, which concerns only the DC servos, the central column of Figure 13 shows that our tests cover the saturation range between 0% and 90% for all DC servo versions. Similarly, as for the actuator saturation, higher percentages of sensor saturation could be reached for inputs with amplitude larger than $A_{\max}$, but are not interesting, since they are known to be outside of the sensor range (equal to $A_{\max}$). From the plots, we can see that the tests exposing sensor saturation (purple markers) are the ones characterised by large amplitudes and low frequencies. Intuitively, at high frequencies, the filtering behaviour (which reduces the amount of input signal that is found in the output and hence reduces its amplitude) prevents the sensor saturation from happening. Furthermore, it is interesting to note that the tests show a sudden transition from tests where the sensor saturation does not appear to tests where the sensor saturation appears to a large extent. This is also confirmed by the boxplots that show a distribution biased toward the lower values of the metric used to quantify the sensor saturation occurrence. Such a behaviour is explained by the control algorithm being unable to bring back the DC servo in a state where the sensor is not saturated once the saturation occurs, therefore keeping it in a saturated state for the rest of the test. Therefore, we can observe that, thanks to the combination of frequency and amplitude coverage, our approach is effective at generating test cases within practically relevant ranges (i.e., smaller than $A_{\max}$) that trigger the saturation of sensors.

Last, we look at the other non-linear phenomena that we inject in the different versions of the DC servo. Those are shown in the right-most column of Figure 13. We observe that the coulomb friction and the input dead-zone and backlash are present in all tests to a high degree. This can be seen both from the boxplots that are all skewed toward the right-hand side of the colorbar but also

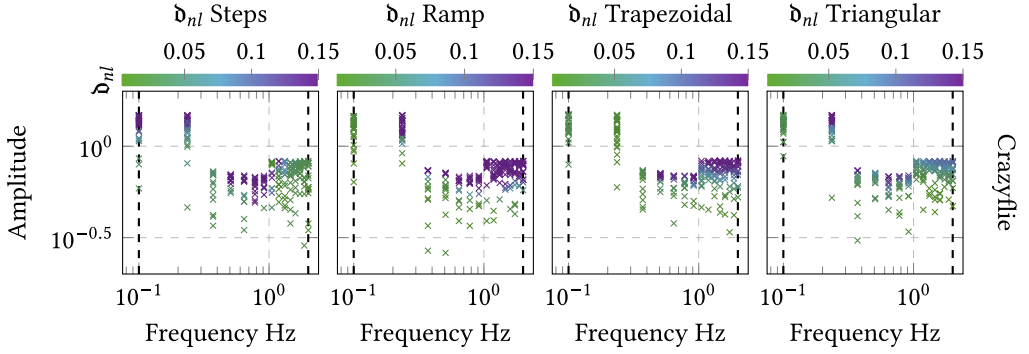


Fig. 14. Degree of non-linearity of the drone test cases divided by shapes. For each test, the main component is plotted on the frequency-amplitude plane. Each point is coloured according to the measured d_{nl} : green corresponds to $d_{nl} = 0$ and purple corresponds to $d_{nl} = t_{nl}$ or greater.

from the scatter plots that are mostly purple. The reason is that such non-linearities affect every movement in the DC servo and therefore are triggered in every test. The only exception is the backlash that appears less in tests characterised by low frequencies (Figure 13, bottom-right plot). In fact, the backlash affects the changes of direction of the input. Hence, for slowly changing signals (low frequencies), there are fewer changes of direction. Differently from the other injected non-linearities, the quadratic friction appears to various degrees across the different tests. In Figure 13, we can see that this variety is achieved mostly by the coverage of the frequency axis. In fact, the colour gradient changes mostly when moving horizontally rather than vertically. Intuitively, friction is a phenomenon related to the rate of change of the system, which is associated to the frequency axis. The quadratic friction is exposed the most around 0.2 Hz and 0.3 Hz, which, as seen in Figure 17, is the closed-loop bandwidth of the system. At this frequency the system is behaving the “fastest,” causing friction to be the largest and hence the difference between quadratic and linear friction to be maximised. We recall that friction is a function of the speed of motion. Above f_b , the input is filtered, and the non-linearity is exposed less. This further exemplifies the relevance of sampling the frequency axis to trigger the different non-linear phenomena.

To summarise, our experiments show that the tests generated by our testing approach effectively trigger the non-linearities present in the SUT. This is done under the constraint of using input values within valid bounds (i.e., smaller than A_{max}). The saturation of actuators and sensors, as well as quadratic friction, are exposed to various degrees. Notably, the tests highlight that such non-linearities appear for specific ranges of frequency and amplitude values, hence providing evidence for the relevance of such characterisation of the inputs when testing a CPS. However, coulomb friction, dead-zone, and backlash are easily triggered in every execution of the SUT in our tests. Such phenomena appear for every movement of the DC servo. Accordingly, they are easily exposed and do not require a dedicated testing approach.

6.4 RQ2—Relevance of Metamorphic Relations

Methodology: We discuss this RQ separately for the different MRs. For each MR, we visually inspect the output of our testing approach (Figures 14–19). We then identify the tests for which the MRs are not satisfied and discuss the causes. For the MRs to be practically relevant, the test cases that do not fulfil them should highlight phenomena that limit the SUT design scope.

Reading Guide for d_{nl} and d_f Plots: We now present figures reporting the d_{nl} and the d_f values computed for the tests executed when applying our testing approach to the SUTs, as well as how

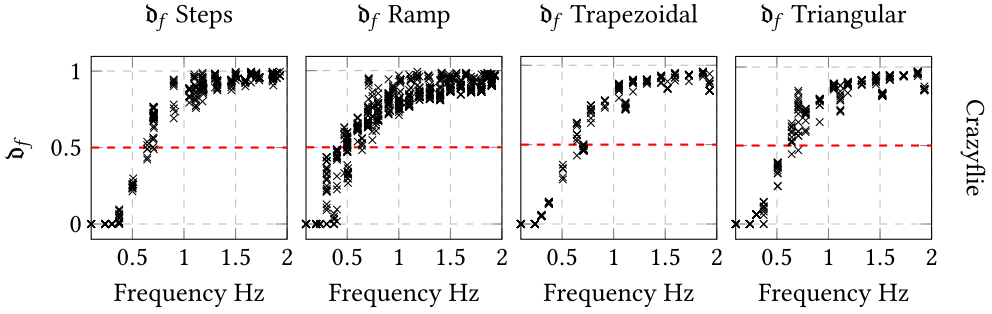


Fig. 15. The figure reports the δ_f values for all frequency-amplitude components of the tests that show linear behaviour (identified with $\delta_{nl} < t_{nl}$). Since MR2 and MR3 do not depend on the test amplitude for tests exposing linear behaviour, we plot the δ_f of each component as a function of their frequency coordinate. The plots are also used to identify the closed-loop bandwidth in the tests made with different shapes. The bandwidth is identified as the frequency where the δ_f becomes larger than the 0.5 threshold, which is highlighted by the red dashed line. Besides some ramp tests most of the results indicate a bandwidth $f_b \approx 0.6$ Hz.

the MRs can be evaluated in such plots. In Figures 14 and 15, we present the plots for the drone. In Figures 16 and 17, we present the plots for the DC servo. In Figures 18 and 19, we present the plots for the lightweight aircraft. The figures are based on the execution of Step 3 of the approach (Section 5.3).

Figures 14, 16, and 18 report the measured δ_{nl} of every executed test. Figure 14 reports the δ_{nl} results for the drone, Figure 16 for the different DC servo versions (where the rows correspond to the different versions of the SUT), Figures 18 for the lightweight aircraft. Since MR1 (the MR concerning the δ_{nl}) discusses tests with the same shape, we separate these plots by shape in the different columns. As suggested in the approach description, we plot the main frequency-amplitude component of each test on the frequency-amplitude plane. This representation is analogous to the non-linear phenomena occurrence plots. However, to distinguish them, we use here a different marker, a cross. In fact, as discussed in Section 4.2.2, the δ_{nl} metric aims to detect the impact of non-linear phenomena on the CPS, not their sole occurrence. Like in Figure 7, we use a logarithmic scale on both axes for readability. For each point, the colouring corresponds to the δ_{nl} value measured in the test. Green corresponds to $\delta_{nl} = 0$ (linear tests within the design scope) and purple corresponds to $\delta_{nl} \geq 0.15$ (non-linear tests outside of the design scope). Different from the non-linear phenomena occurrence plots, the colour gradient is the same in all of these plots. MR1 states that the δ_{nl} value should increase when moving to the right or up in the frequency-amplitude plane. Practically, this corresponds to the colour gradient transitioning from green to purple for increasing amplitudes and frequencies (i.e., when moving away from the origin).

Figures 15, 17, and 19 report the δ_f values measured for all the frequency-amplitude points of the linear tests, for which $\delta_{nl} < t_{nl}$. Figure 15 reports the δ_f values for the drone, Figure 16 for the DC servo (where again the rows correspond to the different versions of the SUT), and Figure 19 for the lightweight aircraft. Since MR2 discusses tests with the same shape and MR3 compares the evaluation of the bandwidth across different shapes, we present the δ_f plots by shape (corresponding again to the different columns). Since MR2 and MR3 do not concern the amplitude, we plot the δ_f value (the vertical coordinate) of each point as a function of its frequency content (the horizontal coordinate). Different from the previous figures, we use here a linear scale on both axes. To ease the identification of the closed-loop bandwidth f_b , we highlight with a red dashed line the 0.5 threshold for the δ_f metric. MR2 states that high frequency content should correspond to the appearance of filtering behaviour. Practically, this corresponds to an increasing δ_f for increasing

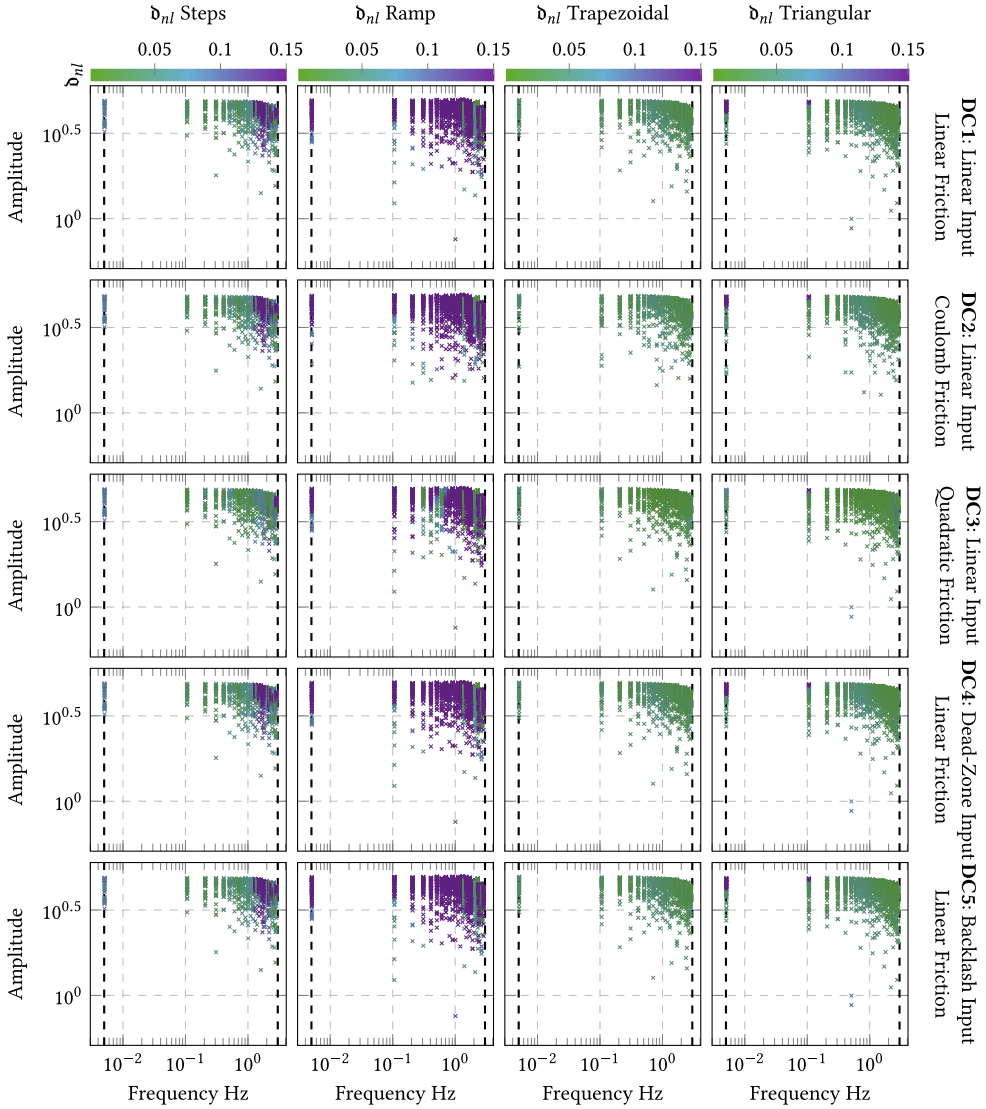


Fig. 16. Evaluation of the d_{nl} metric for the tests with different shapes (the columns) on the different versions of the DC servo (the rows). For each test, we plot the frequency-amplitude main component and colour it according to the measured d_{nl} . The colour gradient has the same interpretation in every plot and goes from green that corresponds to $d_{nl} = 0$ to purple that corresponds to $d_{nl} \geq t_{nl}$.

frequencies. Using the d_f metric, we can also evaluate MR3. To fulfill MR3, the frequency at which the d_f value becomes larger than 0.5 has to be similar across the different shapes.

Results—MR1: For the drone (Figure 14), we observe that MR1 holds for low frequencies (lower than 1 Hz) across the different shapes. However, for frequencies higher than 1 Hz, we observe that an increasing number of tests show linear behaviour (green points) and do not comply with MR1. This appears clearly for tests with steps and triangular shapes, and, to a lesser extent, for the remaining shapes. Despite the quantified occurrence of non-linear phenomena depicted in Figure 12(a) shows that the actuators are fully saturated in that frequency range (practically all

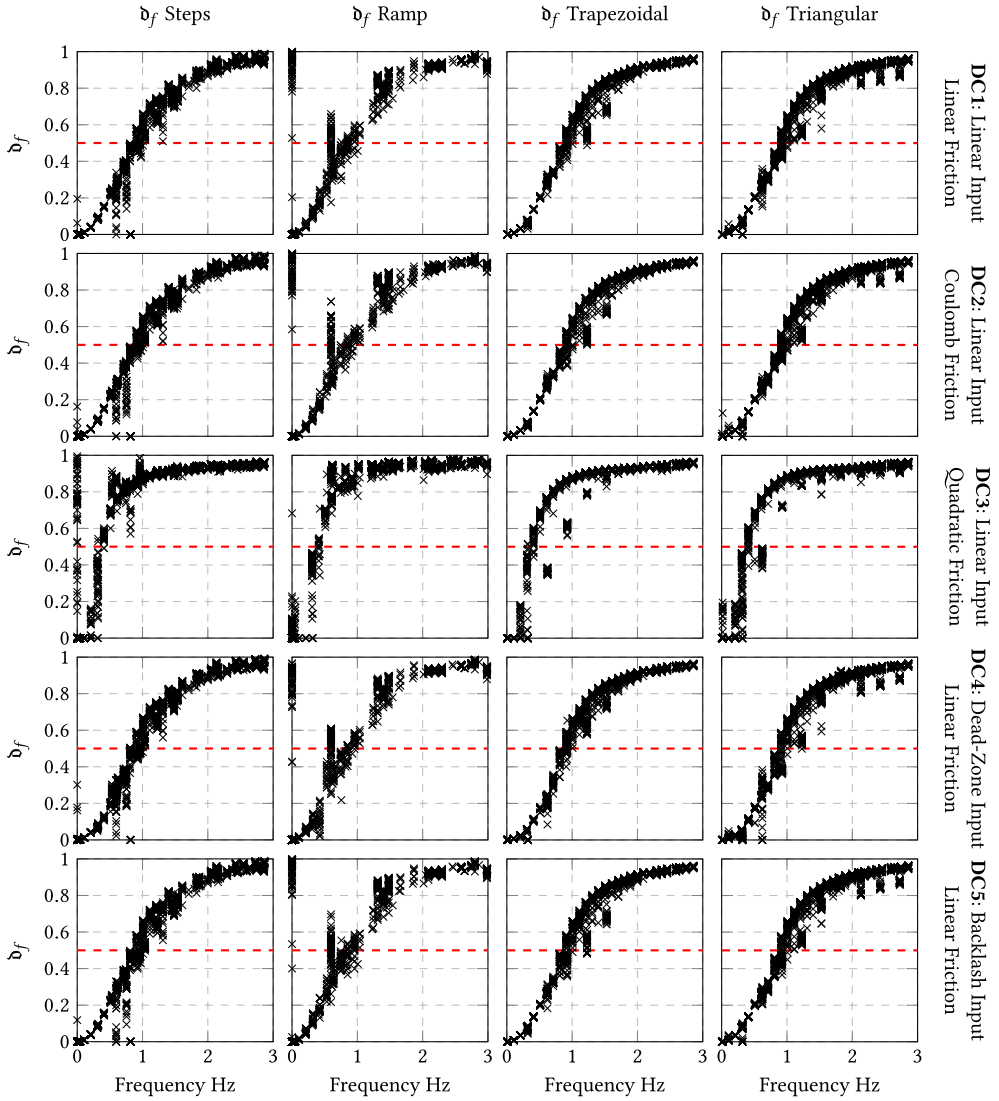


Fig. 17. The figure reports the d_f values for all frequency-amplitude components of the DC servo tests showing linear behaviour (identified with $d_{nl} < t_{nl}$). The tests are separated into different plots according to the considered version of the DC servo (the rows) and the shape used in the tests (the columns). Since MR2 and MR3 do not depend on the test amplitude for tests exposing linear behaviour, we plot the d_f value of each component as a function of their frequency coordinate. The plots are also used to identify the closed-loop bandwidth in the tests made with different shapes. The bandwidth is identified as the frequency where the d_f becomes larger than the 0.5 threshold, which is highlighted by the red dashed line. For most DC servo versions the tests indicate a bandwidth $f_b \approx 0.9$ Hz, the only exception being **DC3** that exposes a bandwidth around $f_b \approx 0.4$ Hz.

markers are purple above 1 Hz), the d_{nl} value in such tests is low. This is due to the filtering action that mitigates the response of the control loop and prevents the non-linearity from introducing new frequency content in the output. This statement can be further verified by observing in Figure 15 that the inputs in the frequency range above 1 Hz all show a d_f value close to 1.

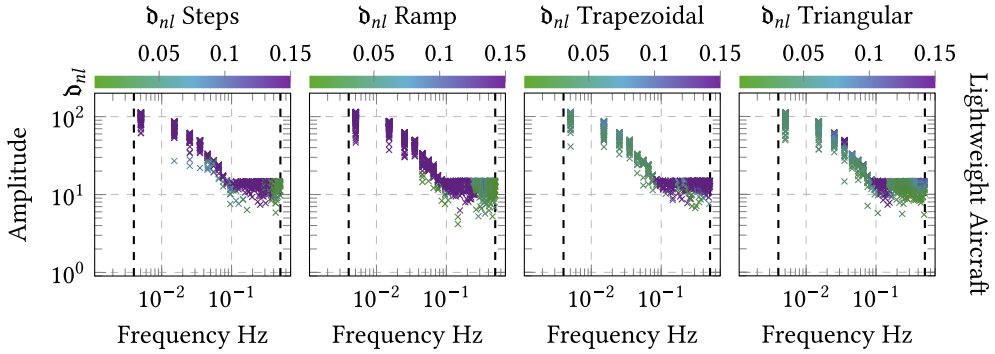


Fig. 18. Evaluation of the d_{nl} metric for the tests with different shapes on the altitude control of the lightweight aircraft. For each test, we plot the frequency-amplitude main component and colour it according to the measured d_{nl} . The colour gradient has the same interpretation in every plot and goes from green that corresponds to $d_{nl} = 0$ to purple that corresponds to $d_{nl} = t_{nl}$.

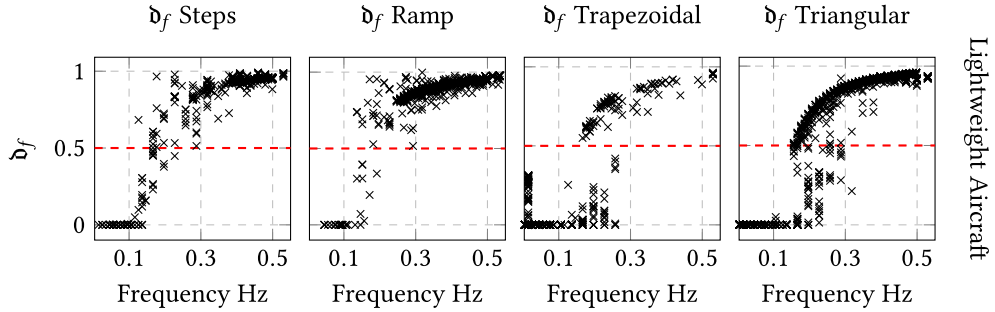


Fig. 19. The figure reports the d_f values for all frequency-amplitude components of the lightweight aircraft tests that show linear behaviour (identified with $d_{nl} < t_{nl}$). Since MR2 and MR3 do not depend on the test amplitude for tests exposing linear behaviour, we plot the d_f of each component as a function of their frequency coordinate. The plots are also used to identify the closed-loop bandwidth in the tests made with different shapes. The bandwidth is identified as the frequency where the d_f becomes larger than the 0.5 threshold, which is highlighted by the red dashed line. The tests indicate a bandwidth $f_b \approx 0.15$ Hz.

Concerning the DC servo (Figure 16), we observe a similar pattern to the one for the drone tests with the steps shape (as well as some of the high-frequency ramp tests). We note that across shapes, except for the trapezoidal tests, tests at the low frequencies and high amplitudes (top-left of the plots) expose a higher d_{nl} value and do not comply with MR1. By comparing with the quantification of the non-linear phenomena occurrence in Figure 13, we can observe that the high d_{nl} value is due to the combination of sensor and actuator saturation (first and second column). As such tests do not comply with MR1, they provide evidence about using d_{nl} for detecting non-linear phenomena that can limit the CPS design scope.

Among the DC servo tests, it is notable that almost all of the ramp tests show high d_{nl} though the proposed metrics, which quantify the occurrence of non-linear phenomena (Figure 13), do not detect a specific non-linear phenomenon. The likely reason for this discrepancy with the other shapes is that the ramp is the shape with the highest number of relevant frequency components (i.e., the frequency-amplitude components selected by Equation (1)). This larger number of relevant frequency components shows that such input exercises more the SUT by feeding more frequencies at the same time to the system. Furthermore, we recall that, in the system, there

are other non-linear phenomena that could not be quantified (i.e., pulse-width modulation and quantisation). Hence, we can suppose that these tests fail because of such phenomena, or a combination of them. However, we note that for lower amplitudes and frequencies some ramp tests yield a lower $\mathfrak{d}_{nl}$. This still partially supports the validity of MR1.

For the lightweight aircraft, we note similar patterns as for the other case studies. First, we observe that most of the tests with steps and ramp shapes show non-linear behaviour. As for the DC servo, this is likely due to the large number of main frequency components that characterise such signals. We can also observe that tests in the low frequency range and low amplitudes show a low $\mathfrak{d}_{nl}$ value. As for DC servo, this supports the validity of MR1. Second, similar to the drone, we also observe that some high-frequency tests retain a lower $\mathfrak{d}_{nl}$ despite the occurrence of actuator saturation (Figure 12(b)). This can be attributed to the filtering action that reduces the stress on the SUT and helps retain linear behaviour.

To conclude, our tests results show that MR1 does not always hold. According to our experiments, the two things that can invalidate MR1 are the unpredictable nature of non-linear phenomena (like the sensor saturation for low frequencies in the DC servo) and the filtering action at high frequencies (as observed for the drone and lightweight aircraft). The falsification of MR1 contradicts the qualitative characterisation in Figure 5. Practically, this implies that a testing strategy leveraging the frequency and amplitude dimensions to expose non-linear behaviours cannot only aim at exploring large amplitude and frequency values. Instead, it is also important to cover the whole input range (e.g., the low frequency range tests in the DC servo that cause sensor saturation). Most notably, we observe that the $\mathfrak{d}_{nl}$ metric is successful in distinguishing the scenarios in which the non-linear phenomena impact the CPS behaviour (hence bringing the CPS out of its design scope) from the scenarios where they do not (e.g., when the filtering action helps retain the linear behaviour despite the actuator saturation).

Results—MR2: To verify if MR2 applies to our tests results, we inspect Figures 15 for the drone, Figure 17 for the DC servo, and Figure 19 for the lightweight aircraft. We observe that, in general, $\mathfrak{d}_f$ shows an increasing value for increasing frequencies, hence fulfilling the MR. The exceptions are some ramp tests that show an high $\mathfrak{d}_f$ for low frequency for all the DC servo versions, as well as some step tests on the DC servo injected with quadratic friction. Furthermore, some trapezoidal low-frequency tests for the lightweight aircraft also show high $\mathfrak{d}_f$ values for low frequency components. Upon manual inspection, we observe for the DC servo that those tests are all characterised by large amplitudes that cause saturation (of both sensors and actuators) to occur. However, in those tests, the output neither follows the reference (hence, the high $\mathfrak{d}_f$) nor exposes new frequency components. For this reason, those tests do not have a high $\mathfrak{d}_{nl}$ and were not excluded by the analysis. For the lightweight aircraft, the trapezoidal tests associated with such points mostly expose a linear tracking behaviour, meaning that they overall follow the input reference. However, these tests also show unexpected deviations during flight, specifically when transitioning from a constantly increasing input to a constant one. Such deviations are not detected by the $\mathfrak{d}_{nl}$ metric as they are small relative to the input amplitude²⁵ but they do reduce in amplitude one of the components in the low frequency range, and are hence detected using the $\mathfrak{d}_f$ metric. Given the limited knowledge that we have about the lightweight aircraft, we are not able to identify the root cause of the failure of such tests. However, it is apparent that such tests show misbehaviour and require further investigation by the engineers who develop the CPS.

Such tests expose a *limitation of the $\mathfrak{d}_{nl}$ metric* regarding its capacity to highlight tests that push the system outside of the control design scope. However, our results show that *the verification of MR2 can compensate for this limitation and be used to complement the $\mathfrak{d}_{nl}$ metric*. In fact, the

²⁵Recall that the metric is normalised over the input amplitude.

verification of MR2 highlights test cases in which the low frequency components (i.e., frequencies below the bandwidth that should be tracked) are not tracked.

Results—MR3: To evaluate MR3, we examine the $\mathfrak{d}_f$ plots for our different case studies. We observe that, for each of the SUTs, the crossing of the 0.5 threshold happens around the same frequency independently of the specific input shape considered for most of the tests. This frequency value is around 0.6 Hz for the drone, just below 1 Hz for all the versions of the DC servo (with the exception of **DC3** where it is below 0.5 Hz²⁶) and around 0.15 Hz for the lightweight aircraft. For the drone, among the various input shapes, the only exceptions are some of the ramp tests that show a bandwidth lower than the other shapes. We identify the tests that invalidate MR3 as those tests that show components with a $\mathfrak{d}_f$ greater than 0.5 at frequencies lower than 0.6 Hz. Upon inspection, we observe that in such tests the drone hits the ground. However, hitting the ground limits the drone's movements but does not introduce any new components in its output. Hence, this phenomenon cannot be detected using the $\mathfrak{d}_{nl}$ metric. This is similar to what we observed in some of the ramp tests for the DC servo when evaluating MR2.

In the case of the lightweight aircraft, we observe that in some trapezoidal and triangular tests the CPS is able to track frequencies higher than the bandwidth of 0.15 Hz. We identify such tests as having a component with $\mathfrak{d}_f$ smaller than 0.5 for frequency higher than 0.2 Hz. When manually inspecting such tests, we observe that they are characterised by small amplitude values (in the order of tens of meters, when the inputs can take values up to several hundreds of meters). This suggests that the ability of the lightweight aircraft to track fast reference inputs can depend on the input amplitude, invalidating MR3 and indicating non-linear behaviour.²⁷ Conversely, for the steps and ramp shapes, we observe some tests that seem to show a lower closed-loop bandwidth. We identify such tests as having a component with $\mathfrak{d}_f$ larger than 0.6 for frequency lower than 0.15 Hz. When manually inspecting such tests, we observe that, similarly to the trapezoidal tests discussed for MR2, they are characterised by the appearance of some undesired behaviours that are however relatively small compared to the input and do not cause the $\mathfrak{d}_{nl}$ value to become larger than 0.15. Therefore, similar to MR2, we can observe that MR3 holds in most of the tests and when it is invalidated, it complements the $\mathfrak{d}_{nl}$ metric in detecting tests where the SUT goes out of the design scope.

To summarise, we observe from our tests that MR1 generally holds but can be invalidated by (1) the filtering action at high frequencies (that increases the robustness of the CPS to non-linear phenomena by reducing their reaction to the reference changes, like for the drone), or (2) by specific non-linear phenomena (that cause unpredictable behaviour of the CPS, like for the sensor saturation in the DC servos).

This latter point supports the use of MR1 for the detection of test cases associated with the appearance of non-linear phenomena specific to the SUT. MR2 and MR3 hold in the majority of our tests. When they do not hold, our tests show that they can be used to complement the $\mathfrak{d}_{nl}$ metric to detect test cases that push the system outside of the design scope. Hence, these MRs compensate for the observed limitations of the $\mathfrak{d}_{nl}$ metric.

6.5 Threats to Validity

We discuss the threats to the validity of our work in terms of external and internal validity. Concerning the external validity, we discuss the generalisability of our observations to other CPSs. We developed the CPS models of the drone and the DC servo used in our case studies, and hence might

²⁶Intuitively, the quadratic friction present in **DC3** makes the DC servo slower.

²⁷Given our limited knowledge of the SUT, we cannot identify the specific phenomenon causing this. Intuitively, in some systems, tracking inputs with small amplitude values is easier than tracking inputs with large amplitude values.

have biased the results. However, we built these CPS models based on the actual documentation of the real implementations of the CPSs. This supports the claim that our case studies are representative of real-world applications. Furthermore, we complement these two case studies with the lightweight aircraft that was independently developed. Notably, the observations that we can draw for this latter case study are the same as for the two previous ones. This further supports the general validity of the findings and their applicability to other CPSs.

Concerning internal validity, we discuss the limitations of our research methodology. It can be noted that we based a significant part of the discussions of the RQs on our own understanding of what the non-linear phenomena that affect our SUTs are, especially for the drone and the DC servo. Since we developed these simulation models based on the actual physical systems (both available to us), we are confident in our complete understanding of the behaviour of these systems. Furthermore, having access to the physical systems allowed us to compare to the actual implementation of the CPSs when uncertain about the model design choices (e.g., the specific quantisation of the sensors and the parameters of the control algorithms).

Finally, it can be noted that we used manual inspections of frequency-amplitude plots to evaluate the satisfaction of our MRs. However, analysing such plots is rather intuitive and helped associate the physical phenomena with the features of the input (e.g., the quadratic friction affecting more the fast-changing inputs). This enabled a deeper understanding of how the non-linear phenomena can limit the design scope of the CPS under test. Nevertheless, an automated inspection method of our MRs is desirable. We leave the development of such method to future work.

6.6 Data Availability

The code of the SUT models and the code implementing the testing approach are available at the following repositories:

- **Crazyflie:** <https://doi.org/10.5281/zenodo.7994669>
- **DC servo and Lightweight Aircraft:** <https://doi.org/10.5281/zenodo.7994698>

These repositories come with instructions to reproduce all of the experiments. Since the tests take some time to execute, we provide the test output traces that can be used to obtain the figures of the article:

- **Crazyflie:** <https://doi.org/10.5281/zenodo.8043446>
- **DC servo:** <https://doi.org/10.5281/zenodo.8043260>
- **Lightweight Aircraft:** <https://doi.org/10.5281/zenodo.8043351>

7 APPROACH APPLICABILITY AND LIMITATIONS

Our experiments show that the proposed testing approach is able to generate stress test cases for individual control loops of control-based CPSs. More specifically, our testing approach generates scenarios that falsify the design assumptions associated with the linearisation of the physics models (RQ1). Furthermore, the proposed metrics (δ_{nl} and δ_f), together with the falsification of the associated MRs, are able to identify the tests in which the non-linear phenomena actually impact the CPS ability to track the input reference. We now discuss the technical aspects that impact the practical applicability of the approach, as well as its applicability to different CPSs.

7.1 Identification of Non-linear Phenomena in Practice

In practical scenarios, evaluating the occurrence of non-linear phenomena (e.g., Figures 12(a) and 13) requires detailed knowledge and understanding of the SUT, which may not always be available. Even more importantly, even if the phenomena are known, it can be hard to know *a priori* which

of them actually limits the CPS design scope. For example, in the DC servo, it cannot be known *a priori* that the sensor and actuator saturation will appear and push the CPS out of its design scope. Conversely, the coulomb friction, the dead-zone, and the backlash non-linearities are triggered in the tests, but they do not affect the CPS design scope. To identify the test cases where non-linear phenomena appear and limit the design scope of the control algorithm, we showed, for our SUTs, that $\mathfrak{d}_{nl}$ and MR1 can be used (RQ2-MR1). Our experiments also showed a limitation in the detection of the design scope based on the quantification of new frequency components in the output (RQ2-MR2 and RQ2-MR3). This is because, in some specific cases, the non-linear phenomena might not introduce large enough new frequency components, causing the $\mathfrak{d}_{nl}$ metric to take small values. However, such tests are detected using MR2 and MR3, since they invalidate the MRs by exposing filtering behaviour at low frequencies (where we would instead expect tracking behaviour) or an inconsistency in the observed closed-loop bandwidth. This shows that the $\mathfrak{d}_f$ metric and the associated MRs can compensate for the limitation of $\mathfrak{d}_{nl}$ and support the detection of test cases that push the SUT out of its design scope.

7.2 Campaign Duration

Concerning test case generation, we note that the proposed rules of thumb for the required input of the approach (Section 5.3, Step 0) generate a high number of test cases (Table 3). To limit the number of tests, Step 1 of the approach aims at reducing the number of test cases needed to cover the input space by restricting the amplitudes ranges to be tested. Across SUTs, this step is effective for the drone and the lightweight aircraft but not for the DC servo versions. In fact, for all the versions of the DC servo, the entire amplitude input range can show linear behaviour, which is a desirable property of the system (see the tests with the trapezoidal and triangular shapes in Figure 16).

The high number of tests can be a limiting factor when considering that we are testing only one control loop of a CPS, which typically has several control loops. In practice, not as many tests are needed, and similar conclusions about the SUT could be drawn with fewer frequency-amplitude points in Figures 14–19. For example, in Figure 16, not as many green and purple points are needed to identify the areas where non-linear phenomena impact the CPS performance. We leave to future work the challenge of minimising the number of test cases needed to stress test a control loop. The problem of minimising the number of test cases can be seen as a testing coverage or stopping criterion problem in the frequency-amplitude plane. In this work, instead, we focus on generating effective test cases that falsify the design assumptions of the SUT. However, as reported above, the execution of the testing campaign for our SUTs took some hours for the drone and the lightweight aircraft, and still no more than a day for the DC servo versions. Therefore, despite the large number of tests, for our case studies, the duration times do not hinder the practical applicability of our testing approach.

7.3 On the Use of Different Shapes

We conclude the technical part of this discussion by noting the difference in results for the different shapes of the tests. This is expected, since these shapes can alter the way the SUT is exercised and hence expose different behaviour. Interestingly, the ramp and step tests more easily expose non-linear behaviour. As mentioned above, this is likely due to the large number of frequency-amplitude components that are included in a ramp and step test. Furthermore, we can also note that there is a similarity between trapezoidal and triangular tests: in fact, they both map to one larger lower-frequency component and a number of higher-frequency components with smaller amplitude. While the investigation of relevant patterns of frequency-amplitude components for the definition of shapes is an interesting aspect that needs further research, we can observe that,

with the proposed shapes, the testing approach already shows promising results regarding the exposition of the CPS design scope.

7.4 General Applicability of the Approach

To evaluate the general applicability of the approach, we discuss how the answers to the RQs change across different SUTs and non-linear phenomena. We note that our case studies include both the same CPS with different non-linear phenomena (the DC servo) as well as the same non-linear phenomenon in different CPS: the actuator saturation in fact appears in each of our CPS. The variety of non-linear phenomena is important, as we want the approach to be able to trigger each of them. The same non-linear phenomenon in different CPS is also important as specific features of the CPS can change its impact on the design scope. Concerning RQ1, across all three case studies, the approach has been able to generate test cases that expose non-linear phenomena. More specifically, saturations and quadratic friction are exposed to various degrees, while the remaining non-linearities are always exposed to a high degree in all tests. The sensor saturation was detected in only a small number of test cases (for all the versions of the DC servo). As discussed, this is possibly a consequence of the choice of the input parameter $A_{\max}$. Increasing this parameter value could increase the number of tests that cause sensor saturation. However, such tests would be “trivially” outside of the DC servo’s design scope, as they correspond to reference values outside of the range of the sensor.²⁸ The input and friction non-linearities, however, are easily triggered in every test. However, as they are not associated with high δ_{nl} or the falsification of the MRs associated with the δ_f metric, we observe that they do not impair the CPS design scope.²⁹

Concerning RQ2, MR1 shows different validity and gives different types of insights across SUTs. For the actuator saturation of the drone and the lightweight aircraft, MR1 is violated because of the robustness introduced by the filtering action. Such robustness also appears in the DC servo studies, although to a smaller degree. For the DC servo, the sensor saturation invalidates MR1, as δ_{nl} increases for decreasing frequencies, specifically, when going from 0.1 to 0.01 Hz in Figure 16. In contrast, the verification of both MR2 and MR3 (as a complement to the δ_{nl}) does not show significant differences across our SUTs. Most notably, the ramp tests that do not satisfy MR2 were similar across all of the SUTs.

To summarise, in our case studies, the effectiveness of our stress test case generation (RQ1) and the validity of MR2 and MR3 do not depend on the specific SUT. However, the verification of MR1 shows differences related to the specific design bounds of the different SUTs. Specifically, it highlights the high-frequency robustness to saturations of the drone and the issues with sensor saturation for low frequency in the DC servo. This is desirable as it helps identify the relevant phenomena that characterise a given SUT. As the results obtained from testing the three SUTs are consistent and promising, we believe our approach can be applied to other SUTs.

7.5 Practical Implications of the Design Scope

We conclude this discussion by addressing the practical implications of the proposed testing approach. We believe that the CPS design scope information provided by our testing approach can be used both for CPS verification and validation, as well as for CPS development. Concerning the verification and validation, the identification of the design scope can be used to distinguish relevant tests for which the requirements need to be evaluated. More practically, if a test is outside of the

²⁸Such test cases should be considered instead for robustness testing (with invalid inputs). In this work, we focus instead on valid inputs.

²⁹It is important to note, however, that such non-linearities (coulomb friction, dead-zone and backlash) can still impair other aspects of system performance, e.g., the accuracy of the reference tracking.

CPS design scope, then it will trivially not fulfil the requirements and hence is not interesting for the verification of the requirements. Such tests can be considered invalid inputs. Furthermore, the design scope information can be used for runtime verification to implement checks on reference sequences that might push the CPS out of its design scope (e.g., large step changes in the case of the lightweight aircraft). Leveraging such runtime checks, the software can either automatically re-shape the signals or raise warnings to the user.

Regarding the development of the CPS, the proposed approach can provide feedback on the design choices that limit the design scope. For example, if the observed design scope does not contain scenarios in which the CPS is expected to perform in a predictable way, then there is a need for re-designing the system to enlarge its scope. Using this information, the engineers can therefore assess if the system achieves the desired performance or if some of its components (e.g., actuators, sensors, or sensor signal filters) need to be re-designed.

8 RELATED WORK

The topic of testing control-based CPS software is not new in the software engineering literature. In fact, control systems are seen as a class of CPSs in which most of the added value is placed in the software part [12]. Accordingly, recent works highlighted interesting research directions at the intersection of control and software engineering [5, 8]. One highlighted direction is the relation between control models and the software implementation of the CPS. Our work explores this direction by testing the CPS implementation to identify the practical validity of the control models (i.e., the design scope).

Testing of control systems has been approached in the literature in the form of model testing, falsification, and runtime verification. Given the widespread industrial use of tools like Mathworks Simulink, model testing has received significant attention in the last decade [9]. Recent work showed the complementarity of model testing and model checking for the verification of CPS requirements [43]. Generation of test traces is one of the main topics in CPS model testing. Very different types of algorithms are used for the generation of input sequences: search algorithms [2, 28, 35–37], classification trees [30], and system-identification-based refinements [39]. We also note a number of application-specific works in the avionics [44, 47, 53] and automotive [10, 11] domains. In contrast to previous work, this article is the first one to propose a test-case generation approach based on a frequency-amplitude description of the input space. Among the previous works on model-testing, we highlight as closely related the work by Matinnejad et al. [38], which puts significant emphasis on the exploration of different values of the first and second derivatives (see Reference [38, Figure 2]). The derivative values are directly connected to the frequency and amplitude dimensions of this article. In fact, the derivative is the ratio between the change in a function input and output: changing the output amplitude or scaling the time automatically (as we do in our test case parametrisation, Equation (4)) directly enables the exploration of different values for the derivatives. As improvement with respect to this previous work, our explicit use of the frequency-amplitude characterisation enables leveraging control-theoretical knowledge and specifically the definition of our proposed qualitative characterisation and MRs.

CPS falsification (as the generation of test cases that falsify a given requirement) is also an active research direction. In a recent work [55], the authors use deep reinforcement learning to perform robustness-guided falsification. Interestingly, they note the importance of making the system internal dynamics available to the reinforcement learning algorithm, which highlights the possible benefits of leveraging control-design information during the testing. Furthermore, a combination of model checking and path planning can be used to invalidate linear-temporal-logic formulas [45]. Similar problems can be addressed with rapid exploration of random trees [20]. With respect to

previous work on the falsification of CPS requirements, in this article, we propose the novel perspective on the falsification of the control design assumptions.

The verification of CPSs has been addressed in the software engineering literature also from the point of view of the runtime verification. Most notably, we remark some works that expand traditional logic (e.g., signal temporal logic) with frequency-domain descriptions of signals under the motivation of verifying data-flow languages (such as Simulink) [13, 19]. Such works are aligned with this article, highlighting the relevance of the frequency-domain description of signals for CPSs and complement it with its application to runtime verification. Another recent work highlights the need to expand the runtime verification theory for its application to CPSs and proposes a new Hybrid Logic of Signals to express desired CPS properties that concern both the cyber and physical components [41]. This logic has been extended to include also source-code-level specifications [17]. To conclude the discussion on runtime verification, we report on a first work that leverages system dynamics (expressed as differential equations, i.e., the same type of models used at control design time) to either terminate the monitoring early, or to skip samples of the signal [1]. While runtime verification is out of the scope of this article, the cited works highlight the relevance of the frequency description of signals, the need to extend software engineering tools for CPSs, and the potential of leveraging control models.

The control and robotics communities show growing awareness of the impact of the software implementation of control algorithms on their performance [49, 57]. Zimmer et al. [57] discuss a case study on the consequences of implementation choices for the control performance. In robotics, software-in-the-loop simulations (i.e., simulations that include the actual control software implementation) can be effective in exposing bugs in the software design of quadcopter controllers [49, 51]. However, such works remain isolated efforts and the leveraging of control theory during the testing process remains limited.

We conclude the discussion of the related literature by discussing the few works that leverage control-theoretical knowledge in the testing of control-based CPSs [2, 26, 39]. In these works, system identification [6] (a field very close to, if not part of, control theory) is used to reduce the number of tests that have to be executed to find faults [39], to reduce the parameter definition effort required by genetic algorithms [2] or to define oracles [26]. In these approaches, system identification is used as part of the testing process itself, and not to obtain knowledge on the SUT. We therefore highlight that none of these works explicitly leverages the control-theoretical design of the SUT or integrates control knowledge in the definition of the software testing problem.

In this article, we are the first to introduce a control engineering perspective in the CPS testing problem definition, with the discussion of the control design assumptions and the resulting limitation of the CPS design scope. Furthermore, we show how to use the frequency domain to (i) perform test case generation for CPS as well as (ii) leverage control-theory knowledge to define metamorphic relations for CPSs. Such contributions extend the state of the art that was mostly characterised by requirement-driven testing approaches and time-domain descriptions of the input space and behaviour of CPSs.

9 CONCLUSION

In this article, we have defined the problem of stress testing control-based, cyber-physical systems (CPS). For this class of CPSs, developed with the use of control theory, we have highlighted the different types of assumptions that the control engineers make during the design of the CPS control layer software. Once the CPS is implemented, such assumptions are not always valid. Accordingly, they can limit the set of scenarios in which the SUT is able to fulfill its requirements. To help engineers identify such scenarios, we have developed a testing approach that pushes the system at the validity boundaries of the linearity design assumptions (i.e., the design assumption

needed by control engineers to use linearised versions of the physical models). We have provided a qualitative characterisation of the CPS control layer input space and proposed a novel test case parametrisation. Leveraging our test case parametrisation, we use the qualitative characterisation to generate test cases that cover the frequency-amplitude plane, propose a novel metric to identify stress test cases, and define metamorphic relations describing the expected CPS behaviour (filtering, linear, non-linear). We have applied the proposed approach to six case studies and evaluated the output of the testing process. Our results show that our testing approach effectively generates test cases that falsify the linearity design assumptions and push the SUT at the bounds of its design scope. Furthermore, the proposed behaviour metrics δ_{nl} and δ_f , together with the metamorphic relations, help identify stress test cases, and especially those outside of the CPS design scope.

Future Work: Future work has to investigate the impact of the use of different input shapes on test case generation. Furthermore, as discussed in Section 7, it may be possible to reduce the number of tests while obtaining similar results. Hence, improved sampling strategies (e.g., adaptive ones [15]) should be investigated. Another technical extension of this work should involve developing a method to assess whether a test, which does not necessarily consist of a repeated input sequence, belongs to the design scope of the CPS or not. More specifically, this would require a study on the generalisability of the proposed metrics and MRs in handling arbitrary signals.

On a higher level, the main contribution of this work is to open a new perspective on the testing of control-based CPSs: stress testing driven by control design assumptions. In future work, we plan to extend this approach to the testing of other assumptions not considered here, such as not accounting for the control mode changes and the interaction between different control loops. Furthermore, another important research direction is testing the combined falsification of the different assumptions, e.g., when the falsification of the linearised model is combined with the falsification of the mode changes, and investigate how such combinations affect the design scope of the SUT.

ACKNOWLEDGMENTS

At the time of doing this work, Claudio Mandrioli and Martina Maggio were members of ELLIIT Strategic Research Area at Lund University.

REFERENCES

- [1] Houssam Abbas and Borzoo Bonakdarpour. 2022. Leveraging system dynamics in runtime verification of cyber-physical systems. In *Proceedings of the 11th International Symposium on Verification Principles: Leveraging Applications of Formal Methods, Verification and Validation (ISoLA'22)*. Springer-Verlag, Berlin, 264–278. https://doi.org/10.1007/978-3-031-19849-6_16
- [2] Aldeida Aleti and Lars Grunske. 2015. Test data generation with a kalman filter-based adaptive genetic algorithm. *J. Syst. Softw.* 103 (2015), 343–352. <https://doi.org/10.1016/j.jss.2014.11.035>
- [3] Karl Johan Astrom and Richard M. Murray. 2008. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, Princeton, NJ.
- [4] Jon Ayerdi, Valerio Terragni, Aitor Arrieta, Paolo Tonella, Goiuria Sagardui, and Maite Arratibel. 2021. Generating metamorphic relations for cyber-physical systems with genetic programming: An industrial case study. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE'21)*. ACM, New York, NY, 1264–1274. <https://doi.org/10.1145/3468264.3473920>
- [5] Balaji Balasubramaniam, Hamid Bagheri, Sebastian Elbaum, and Justin Bradley. 2020. Investigating controller evolution and divergence through mining and mutation. In *Proceedings of the ACM/IEEE 11th International Conference on Cyber-Physical Systems (ICCPs'20)*. IEEE, 151–161. <https://doi.org/10.1109/ICCPs48487.2020.00022>
- [6] Sergio Bittanti. 2019. *Model Identification*. John Wiley and Sons, Chapter 4, 81–105. Retrieved from <https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781119546405.ch4>. <https://doi.org/10.1002/9781119546405.ch4>
- [7] Sergey Bozhko, Georg von der Brüggen, and Björn B. Brandenburg. 2021. Monte carlo response-time analysis. In *Proceedings of the IEEE Real-Time Systems Symposium (RTSS'21)*. IEEE, 342–355. <https://doi.org/10.1109/RTSS52674.2021.00039>

- [8] Justin M. Bradley and Hamid Bagheri. 2020. Control software: Research directions in the intersection of control theory and software engineering. In *Proceedings of the AIAA Scitech Forum*. Aerospace Research Center. Retrieved from <https://arc.aiaa.org/pdf/10.2514/6.2020-2102>. <https://doi.org/10.2514/6.2020-2102>
- [9] Lionel Briand, Shiva Nejati, Mehrdad Sabetzadeh, and Domenico Bianculli. 2016. Testing the untestable: Model testing of complex software-intensive systems. In *Proceedings of the 38th International Conference on Software Engineering Companion (ICSE '16)*. ACM, New York, NY, 789–792. <https://doi.org/10.1145/2889160.2889212>
- [10] Eckard Bringmann and Andreas Krämer. 2006. Systematic testing of the continuous behavior of automotive systems. In *Proceedings of the International Workshop on Software Engineering for Automotive Systems (SEAS'06)*. ACM, New York, NY, 13–20. <https://doi.org/10.1145/1138474.1138479>
- [11] Eckard Bringmann and Andreas Krämer. 2008. Model-based testing of automotive systems. In *Proceedings of the 1st International Conference on Software Testing, Verification, and Validation*. IEEE, 485–493. <https://doi.org/10.1109/ICST.2008.45>
- [12] M. Broy, I. H. Kruger, A. Pretschner, and C. Salzmann. 2007. Engineering automotive software. *Proc. IEEE* 95, 2 (Feb. 2007), 356–373. <https://doi.org/10.1109/JPROC.2006.888386>
- [13] Aleksandar Chakarov, Sriram Sankaranarayanan, and Georgios Fainekos. 2012. Combining time and frequency domain specifications for periodic signals. In *Runtime Verification*, Sarfraz Khurshid and Koushik Sen (Eds.). Springer, Berlin, 294–309.
- [14] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. 2018. Meta-morphic testing: A review of challenges and opportunities. *ACM Comput. Surv.* 51, 1, Article 4 (Jan. 2018), 27 pages. <https://doi.org/10.1145/3143561>
- [15] Tsong Yueh Chen, Hing Leung, and Ieng Kei Mak. 2005. Adaptive random testing. In *Proceedings of the 9th Asian Computing Science Conference on Advances in Computer Science*. Springer, Berlin, 320–329. https://doi.org/10.1007/978-3-540-30502-6_23
- [16] James W. Cooley and John W. Tukey. 1965. An algorithm for the machine calculation of complex fourier series. *Math. Comp.* 19 (1965), 297–301.
- [17] Joshua Heneage Dawes and Domenico Bianculli. 2022. Specifying source code and signal-based behaviour of cyber-physical system components. In *Proceedings of the 18th International Conference on Formal Aspects of Component Software (FACS'22)*. Springer-Verlag, Berlin, 20–38. https://doi.org/10.1007/978-3-031-20872-0_2
- [18] Lane Desborough and Randy Miller. 2002. Increasing Customer Value of Industrial Control Performance Monitoring—Honeywell's Experience. <https://core.ac.uk/display/102313613>
- [19] Alexandre Donzé, Oded Maler, Ezio Bartocci, Dejan Nickovic, Radu Grosu, and Scott Smolka. 2012. On temporal logic and signal processing. In *Automated Technology for Verification and Analysis*, Supratik Chakraborty and Madhavan Mukund (Eds.). Springer, Berlin, 92–106.
- [20] Tommaso Dreossi, Thao Dang, Alexandre Donzé, James Kapinski, Xiaoqing Jin, and Jyotirmoy V. Deshmukh. 2015. Efficient guiding strategies for testing of temporal properties of hybrid systems. In *NASA Formal Methods*, Klaus Havelund, Gerard Holzmann, and Rajeev Joshi (Eds.). Springer International Publishing, Cham, 127–142.
- [21] Bineet Ghosh, Clara Hobbs, Shengjie Xu, Parasara Sridhar Duggirala, James H. Anderson, P. S. Thiagarajan, and Samarjit Chakraborty. 2022. Statistical hypothesis testing of controller implementations under timing uncertainties. In *Proceedings of the IEEE 28th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA'22)*. IEEE, 11–20. <https://doi.org/10.1109/RTCSA55878.2022.00008>
- [22] J. C. Gille-Maisani and P. Decaulne. 1959. *Feedback Control Systems: Analysis, Synthesis, and Design*. McGraw-Hill. Retrieved from <https://books.google.se/books?id=9WG9zQEACAAJ>
- [23] Marcus Greiff. 2017. Modelling and Control of the Crazyfly Quadrotor for Aggressive and Autonomous Flight by Optical Flow Driven State Estimation. Student Paper. <https://lup.lub.lu.se/student-papers/search/publication/8905295>
- [24] Marcus Greiff, Patrik Persson, Zhiyong Sun, Karl Åström, and Anders Robertsson. 2021. Quadrotor Control on $SU(2) \times R^3$ with SLAM Integration. <https://doi.org/10.48550/ARXIV.2110.01099>
- [25] Xiao He, Xingwei Wang, Jia Shi, and Yi Liu. 2020. Testing high performance numerical simulation programs: Experience, lessons learned, and open issues. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA'20)*. ACM, New York, NY, 502–515. <https://doi.org/10.1145/3395363.3397382>
- [26] Zhijian He, Yao Chen, Enyan Huang, Qixin Wang, Yu Pei, and Haidong Yuan. 2019. A system identification based oracle for control-CPS software fault localization. In *Proceedings of the IEEE/ACM 41st International Conference on Software Engineering (ICSE'19)*. IEEE, 116–127. <https://doi.org/10.1109/ICSE.2019.00029>
- [27] Tingshu Hu and Zongli Lin. 2001. *Introduction*. Birkhäuser Boston, Boston, MA, 1–9. https://doi.org/10.1007/978-1-4612-0205-9_1
- [28] Joachim Hänsel, Daniela Rose, Paula Herber, and Sabine Glesner. 2011. An evolutionary algorithm for the generation of timed test traces for embedded real-time systems. In *Proceedings of the 4th IEEE International Conference on Software Testing, Verification and Validation*. IEEE, 170–179. <https://doi.org/10.1109/ICST.2011.37>

- [29] H. K. Khalil. 2002. *Nonlinear Systems*. Prentice Hall. Retrieved from https://books.google.se/books?id=t_d1QgAACAAJ
- [30] Klaus Lamberg, Michael Beine, Mario Eschmann, Rainer Otterbach, Mirko Conrad, and Ines Fey. 2004. Model-based testing of embedded automotive software using mtest. In *Proceedings of the SAE World Congress and Exhibition*. SAE International, 11 pages. <https://doi.org/10.4271/2004-01-1593>
- [31] Edward A. Lee. 2015. The past, present and future of cyber-physical systems: A focus on models. *Sensors* 15, 3 (2015), 4837–4869. <https://doi.org/10.3390/s150304837>
- [32] Yue Lu, Thomas Nolte, Iain Bate, and Liliana Cucu-Grosjean. 2012. A statistical response-time analysis of real-time embedded systems. In *Proceedings of the IEEE 33rd Real-time Systems Symposium*. IEEE, 351–362. <https://doi.org/10.1109/RTSS.2012.85>
- [33] Gabriele Magnani, Daniele Cattaneo, Michele Chiari, and Giovanni Agosta. 2021. The impact of precision tuning on embedded systems performance: A case study on field-oriented control. In *Proceedings of the 12th Workshop on Parallel Programming and Run-time Management Techniques for Many-core Architectures and 10th Workshop on Design Tools and Architectures for Multicore Embedded Computing Platforms (PARMA-DITAM'21) (Open Access Series in Informatics (OASIs), Vol. 88)*, João Bispo, Stefano Cherubin, and José Flich (Eds.). Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 3:1–3:13. <https://doi.org/10.4230/OASIs.PARMA-DITAM.2021.3>
- [34] Claudio Mandrioli, Nyberg Carlsson Max, and Martina Maggio. 2023. Testing abstractions for cyber-physical control systems. *ACM Trans. Softw. Eng. Methodol.* 1, 1, Article 1 (Aug. 2023), 31 pages. <https://doi.org/10.1145/3617170>
- [35] Bogdan Marculescu, Robert Feldt, Richard Torkar, and Simon Poulding. 2015. An initial industrial evaluation of interactive search-based testing for embedded software. *Appl. Soft Comput.* 29 (Apr. 2015), 26–39. Issue 0. <https://doi.org/10.1016/j.asoc.2014.12.025>
- [36] Reza Matinnejad, Shiva Nejati, Lionel Briand, and Thomas Bruckmann. 2014. MiL testing of highly configurable continuous controllers: Scalable search using surrogate models. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE'14)*. ACM, New York, NY, 163–174. <https://doi.org/10.1145/2642937.2642978>
- [37] Reza Matinnejad, Shiva Nejati, and Lionel C. Briand. 2017. Automated testing of hybrid simulink/stateflow controllers: Industrial case studies. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE'17)*. ACM, New York, NY, 938–943. <https://doi.org/10.1145/3106237.3117770>
- [38] Reza Matinnejad, Shiva Nejati, Lionel C. Briand, and Thomas Bruckmann. 2016. Automated test suite generation for time-continuous simulink models. In *Proceedings of the IEEE/ACM 38th International Conference on Software Engineering (ICSE'16)*. ACM, 595–606. <https://doi.org/10.1145/2884781.2884797>
- [39] Claudio Menghi, Shiva Nejati, Lionel C. Briand, and Yago Isasi Parache. 2019. Approximation-refinement testing of compute-intensive cyber-physical models: An approach based on system identification. Retrieved from <http://arxiv.org/abs/1910.02837>
- [40] Claudio Menghi, Shiva Nejati, Khoulood Gaaloul, and Lionel C. Briand. 2019. Generating automated and online test oracles for simulink models with continuous and uncertain behaviors. In *Foundations of Software Engineering*. ACM, 27–38.
- [41] Claudio Menghi, Enrico Viganò, Domenico Bianculli, and Lionel C. Briand. 2021. Trace-checking CPS properties: Bridging the cyber-physical gap. In *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering (ICSE'21)*. ACM, 847–859. <https://doi.org/10.1109/ICSE43902.2021.00082>
- [42] Mark W. Mueller, Markus Hehn, and Raffaello D'Andrea. 2016. Covariance correction step for Kalman filtering with an attitude. *J. Guid. Control Dynam.* 40, 9 (2016), 1–7.
- [43] Shiva Nejati, Khoulood Gaaloul, Claudio Menghi, Lionel C. Briand, Stephen Foster, and David Wolfe. 2019. Evaluating model testing and model checking for finding requirements violations in simulink models. In *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE'19)*. ACM, New York, NY, 1015–1025. <https://doi.org/10.1145/3338906.3340444>
- [44] Jan Peleska. 2002. Hardware/software integration testing for the new airbus aircraft families. Retrieved from <http://www.informatik.uni-bremen.de/agbs/jp/papers/peleskaTestCom2002.html>. https://doi.org/10.1007/978-0-387-35497-2_24
- [45] Erion Plaku, Lydia E. Kavrakı, and Moshe Y. Vardi. 2009. Falsification of LTL safety properties in hybrid systems. In *Tools and Algorithms for the Construction and Analysis of Systems*, Stefan Kowalewski and Anna Philippou (Eds.). Springer, Berlin, 368–382.
- [46] Kshirasagar Naik Priyadarshi Tripathy. 2008. *Acceptance Testing*. John Wiley and Sons, Chapter 14, 450–470. Retrieved from <https://onlinelibrary.wiley.com/doi/pdf/10.1002/9780470382844.ch14> <https://doi.org/10.1002/9780470382844.ch14>
- [47] Tariq Samad and Gary Balas. 2003. *Frontmatter*. John Wiley and Sons, i–xx. <https://doi.org/10.1002/047172288X.fmatter>
- [48] Alex Sanchez-Stern, Pavel Panchekha, Sorin Lerner, and Zachary Tatlock. 2018. Finding root causes of floating point error. *SIGPLAN Not.* 53, 4 (June 2018), 256–269. <https://doi.org/10.1145/3296979.3192411>

- [49] Giuseppe Silano, Emanuele Aucone, and Luigi Iannelli. 2018. CrazyS: A software-in-the-loop platform for the crazyflie 2.0 nano-quadcopter. In *Proceedings of the 26th Mediterranean Conference on Control and Automation (MED'18)*. IEEE, 1–6. <https://doi.org/10.1109/MED.2018.8442759>
- [50] S. Sinharay. 2010. Continuous probability distributions. In *International Encyclopedia of Education* (3rd ed.), Penelope Peterson, Eva Baker, and Barry McGaw (Eds.). Elsevier, Oxford, 98–102. <https://doi.org/10.1016/B978-0-08-044894-7.01720-6>
- [51] C. S. Timperley, A. Afzal, D. S. Katz, J. M. Hernandez, and C. Le Goues. 2018. Crashing simulated planes is cheap: Can simulation detect robotics bugs early? In *Proceedings of the IEEE 11th International Conference on Software Testing, Verification and Validation (ICST'18)*. IEEE, 331–342. <https://doi.org/10.1109/ICST.2018.00040>
- [52] Nils Vreman, Anton Cervin, and Martina Maggio. 2021. Stability and performance analysis of control systems subject to bursts of deadline misses. In *Proceedings of the 33rd Euromicro Conference on Real-Time Systems (ECRTS'21)*, Vol. 196. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, Germany, 23 pages. <https://doi.org/10.4230/LIPIcs.ECRTS.2021.15>
- [53] A. L. White. 2001. Comments on modified condition/decision coverage for software testing [of flight control software]. In *Proceedings of the IEEE Aerospace Conference Proceedings (Cat. No.01TH8542)*, Vol. 6. IEEE, 2821–2827. <https://doi.org/10.1109/AERO.2001.931302>
- [54] Meng Wu, Haibo Zeng, Chao Wang, and Huafeng Yu. 2017. INVITED: Safety guard: Runtime enforcement for safety-critical cyber-physical systems. In *Proceedings of the 54th ACM/EDAC/IEEE Design Automation Conference (DAC'17)*. ACM, 1–6. <https://doi.org/10.1145/3061639.3072957>
- [55] Yoriyuki Yamagata, Shuang Liu, Takumi Akazaki, Yihai Duan, and Jianye Hao. 2021. Falsification of cyber-physical systems using deep reinforcement learning. *IEEE Trans. Softw. Eng.* 47, 12 (2021), 2823–2840. <https://doi.org/10.1109/TSE.2020.2969178>
- [56] Xin Yi, Liqian Chen, Xiaoguang Mao, and Tao Ji. 2017. Efficient global search for inputs triggering high floating-point inaccuracies. In *Proceedings of the 24th Asia-Pacific Software Engineering Conference (APSEC'17)*. IEEE, 11–20. <https://doi.org/10.1109/APSEC.2017.7>
- [57] Michael Zimmer, J. Hedrick, and Edward A. Lee. 2015. Ramifications of software implementation and deployment: A case study on yaw moment controller design. In *Proceedings of the American Control Conference (ACC'15)*. 2014–2019.

Received 15 December 2022; revised 11 August 2023; accepted 16 August 2023